视觉-文本大模型 + 机器学习 / 卷积网络 八股问答

> 与《八股知识问答 118 问》的分工：那份是**多模态总览 + 项目接口话术**，这份是**VLM 深度知识 + 机器学习算法原理推导**，两份不重复、可对照看。 编号用 A（视觉-文本大模型 + 损失/架构/LoRA，77 问）、B（机器学习算法，45 问）、C（图像处理与卷积网络 + 视觉基础模型 + 单目深度 + 激活函数，88 问），共 **210 问**，避免与 Q1~Q118 冲突。 **§ B6 附原理验算代码**（47 项断言，面向 Python 语法不熟者逐行注释）。 难度：★★★ 几乎必问 · ★★ 大概率问 · ★ 加分题。

Part A 视觉-文本大模型（VLM）

A1 模型族谱与架构（A1–A12）

A1 ★★★ VLM 有哪几种架构范式？各自的代表模型和取舍？

A：四种范式，按"视觉怎么进语言模型"分：

范式	结构	代表	取舍
① 双塔对比（encoder-only）	图像塔 + 文本塔，各自编码后算相似度	CLIP、ALIGN、SigLIP	检索/零样本分类强，**不能生成**，只学全局对齐
② 融合编码器	图文进同一个 Transformer（cross-attention 交互），输出融合表征	ALBEF、BLIP、ViLT	适合匹配/理解类任务，参数量比双塔小，但仍不生成自由文本
③ 编码器-解码器桥接（主流 VLM）	冻结/半冻结视觉塔 → 桥接层 → LLM 自回归生成	BLIP-2、LLaVA、Flamingo、Qwen-VL、InternVL	**能力最强、可指令化**，代价是 token 多、训练分阶段
④ 原生/早期融合（Early fusion）	一开始就把图像 patch 与文本 token 拼进同一个序列，统一自回归	Chameleon、Emu3、Fuyu	结构最简洁、天然支持交错图文，但需从零训或大改，工程门槛高

面试答法："主流是第 ③ 种——因为它的训练成本最低（可冻结双塔）、能力上限又最高；第 ④ 种是方向但需要从头训，数据门槛太高。"

追问：Fuyu 有什么特别？→ **没有独立视觉编码器**，直接把图像 patch 线性投影成 token 送进语言模型，省掉视觉塔、也省掉桥接层，代价是需要更强数据、分辨率灵活性靠 patch 切分。

A2 ★★★ LLaVA、BLIP-2、Flamingo 三者的桥接设计有什么本质区别？

A：

	桥接方式	视觉 token 数量	训练成本	核心思想
LLaVA	2 层 MLP（线性投影）	**不压缩**（与 patch 数相同）	最低（只训 MLP）	视觉塔特征已够好，简单映射即可
BLIP-2	**Q-Former**（32/64 个可学习 query + cross-attention）	**固定**（32/64）	中（两阶段多任务）	压缩 + 强对齐
Flamingo	Perceiver Resampler + **LLM 里插 gated cross-attention**	固定（resampler 输出）	中（插入新层）	不改 LLM 主干权重，只加"旁路"

本质区别一句话：LLaVA 是"**改维度**"，BLIP-2 是"**压数量 + 强对齐**"，Flamingo 是"**在 LLM 内部做融合**"。

追问：为什么 LLaVA 敢用最笨的 MLP？→ 因为它冻结双塔、数据量大（数十万~百万级指令），MLP 已经足够；而且论文消融显示，在冻结设定下复杂桥接的收益不显著。**这是"简单方案 + 足够数据"胜过"精巧结构"的典型案例。**

A3 ★★★ 视觉塔为什么大多用 CLIP/SigLIP 而不是 ImageNet 预训练的 ResNet/ViT?

A：三条理由：

1. **语义空间已经和文本对齐**：CLIP 系在 4 亿级图文对上做过对比学习，特征里已经含"可被语言描述"的语义；ImageNet 特征是判别 1000 类的，语义粒度不匹配。
2. **冻结就能用**：CLIP 特征在冻结条件下零样本泛化好，适合 VLM"冻结视觉塔"的训练范式。
3. **ViT 结构同构**：patch token 序列可以直接送进 LLM，不需要额外适配。

什么时候不用 CLIP：需要**密集预测**（分割/深度/占用）时，CLIP 的全局对齐特征反而丢失细粒度空间信息，这时 DINOv2（自蒸馏，保留空间结构）或纯监督 ViT 更合适。**混合做法**：CLIP 塔给语义 + DINOv2 给结构，双塔拼接。

追问：SigLIP 比 CLIP 好在哪？→ 用 pairwise sigmoid 替代 softmax 交叉熵，**不需要全 batch 归一化**，因此 batch 可以小、能分布式异步训练，更省内存；效果同级甚至更好，是现在很多开源 VLM 的默认视觉塔。

A4 ★★ BLIP 的 CapFilt 是干什么的?

A：网页图文对（alt-text）噪声大——图里是"一只狗在草地上"，alt-text 却是"点击购买"。CapFilt = **Captioner + Filter**：

1. **Captioner**：用图像生成 caption（合成描述）；
2. **Filter**：用 ITM 头判断"图文是否匹配"，把原 alt-text 和合成 caption 里不匹配的过滤掉。

本质是用模型给自己造更干净的监督（bootstrapping）。这条和现在"用强 VLM 重新 caption 训练数据"（ShareGPT4V 一类）是同一思想。

A5 ★★ ALBEF 的三个训练目标是什么？为什么需要 ITM?

A：**ITC（图文对比）+ ITM（图文匹配）+ MLM（掩码语言建模）**。

- **ITC** 学全局粗对齐（快、但只能到"整体像不像"）；
- **ITM** 用融合后的特征做二分类（细粒度、但要负样本，靠 ITC 的相似度做 **hard negative 挖掘**）；
- **MLM** 让模型学词级与图像区域的对应（补细粒度）。

为什么需要 ITM：ITC 用的是各自独立的编码（没有跨模态交互），学不到"这个词对应这块区域"；ITM 让图文进同一个编码器交互。**代价是推理时每对都要过一次融合编码器，慢——所以 BLIP-2 之后就用生成式目标替代了它。**

A6 ★★ CoCa 和 BLIP 的思路差异?

A：CoCa 把"**对比学习**（对齐）"和"**caption 生成**（生成）"放在**一个模型的两个头**上，共享图像编码器：一个头做图文对比（学判别式表征），另一个头做自回归 caption（学生成能力）。

意义：**证明对齐与生成可以共存且互补**（对比目标提供稳定的语义空间，生成目标提供细粒度语言监督）。现在很多 VLM 训练都是这个混合思路的延伸。

A7 ★★★ 视觉 token 到底有多少？为什么它是 VLM 的"预算中枢"?

A：算一遍就有体感： - 336×336、patch14 → 24×24 = **576 token** - 448×448、patch14 → 32×32 = **1024 token** - LLaVA-1.6 的 anyres（4 切片 + 1 缩略图）→ 约 **2880 token** - InternVL 的 pixel shuffle（2×2 拼通道）→ token **÷4**

而一张 4096 上下文的 LLM，一张高分辨率图就能吃掉 70%。所以：**分辨率（看得清）↔ token 数（算力/显存）↔ 上下文（能塞几张图/多轮）三者是硬约束的三角**。

工程手段：pixel shuffle、池化、Q-Former 压缩、动态分辨率（按需给 token）、只取关键帧（视频）。

A8 ★★ Qwen-VL 的 M-RoPE 解决什么问题?

A：普通 RoPE 是 1D 的（按序列位置旋转），但视觉 token 有 **时间（帧）+ 高 + 宽** 三维结构。M-RoPE（Multimodal RoPE）把旋转维度切成三份，分别编码 t/h/w，于是： - 同一帧内相邻 patch 的相对位置正确； - 不同帧之间有时间相对关系； - 图像与文本混排时位置语义不乱。

意义：让一个模型同时处理**图像、多图、视频**而不需要改结构——这是"原生多模态位置编码"的代表做法。

A9 ★ InternVL 的 pixel shuffle 为什么能省 token?

A：把相邻 2×2 个 patch 在**通道维**拼起来：原本 4 个 patch 各 `d` 维 → 合成 1 个 patch `4d` 维。token 数 ÷4，再用一个 MLP 把 `4d` 投回 `d`。

代价：空间分辨率信息被"压缩进通道"，细粒度定位会略损；但配合**动态分辨率**（大图切块）能补回来。这是"用通道换序列长度"的典型取舍。

A10 ★★ MiniGPT-4 和 LLaVA 的差别是什么?

A：结构上 MiniGPT-4 用 **Q-Former + 单层线性投影**（沿用 BLIP-2 的视觉侧），LLaVA 用 **CLIP ViT + 2 层 MLP**。数据上 MiniGPT-4 用短 caption 预训练 + 高质量长对话微调（用 GPT-4 生成、人工清洗），LLaVA 用 GPT-4 生成的视觉指令数据。

历史意义：MiniGPT-4 证明了"冻结 LLM + 轻量桥接"能获得对话式视觉能力；LLaVA 把这条路线做到了极致简化 + 规模化，成为开源 VLM 的事实标准。

A11 ★ 原生多模态（Early fusion）和"桥接式"到底差在哪?

A：差在**融合发生的位置**和**训练方式**：

- 桥接式：视觉塔独立编码（可能冻结）→ 投影 → 拼进 LLM 序列。融合发生在 LLM 的第一层之后（视觉 token 已经是"外来词"）。
- 原生式：图像 patch 直接当 token，与文本 token 从第一层起就在同一序列里做 attention。**模型从一开始就不知道"这是图像"还是"这是文本"**，全靠数据学。

代价：原生式通常要从头预训练（或大规模继续预训练），数据门槛极高；好处是**交错图文（多图穿插在长文档里）**天然支持，且不会出现"视觉塔与 LLM 表征错配"。

A12 ★ 视觉塔参数量要不要大？为什么很多模型用 ViT-L 而不是 ViT-H/G?

A：这是**性价比**问题。视觉塔只在 prefill 时跑一次，但它的计算量与**分辨率平方 × 参数量**挂钩；而 LLM 的算力主要在 decode（逐 token）。所以在"视觉塔够用"的前提下，加参数到视觉塔的边际收益远低于加在 LLM 上。

实践：ViT-L（约 300M）是主流；追求更强视觉理解时才上 ViT-H/SoViT；端侧反过来会用 ViT-S/B 配蒸馏。

A2 训练范式与数据（A13–A22）

A13 ★★★ VLM 训练为什么必须分阶段？一次性端到端训行不行?

A：能训，但**代价大且不稳**，原因三条：

1. **初始化错配**：随机初始化的桥接层输出是噪声，若同时解冻 LLM，这些噪声会通过梯度污染 LLM 的预训练能力（灾难性遗忘）。
2. **学习率不可共享**：视觉塔需要小 lr（保护预训练特征），桥接层需要大 lr（从零学），LLM 需要中 lr。分阶段可以分别设。
3. **数据配比不同**：对齐阶段用海量短 caption（学"图上有啥"），指令阶段用少量高质量对话（学"怎么答"）。混在一起训，模型会偏向数据量大的那类。

标准做法：阶段一冻结双塔只训桥接（对齐）→ 阶段二解冻 LLM 做指令微调 → 阶段三（可选）偏好对齐。

A14 ★★★ 对齐阶段（stage-1）为什么要冻结 LLM?

A：三个原因：

1. **保护语言能力**：桥接层初始输出没有语义，让 LLM 去适配它 = 让 LLM 学"如何理解噪声"，会破坏语言先验。
2. **训练信号更干净**：冻结 LLM 后，梯度只能改桥接层，等于在解"如何把视觉特征翻译成 LLM 能懂的词"这个子问题。
3. **成本**：只训几百万参数，单卡就能跑。

追问：那为什么有些模型（如 InternVL）在阶段一就解冻部分 LLM？→ 数据量足够大时（亿级图文对），直接联合训练能获得更强的对齐；小数据下必须冻结，否则一定崩。

A15 ★★ 指令微调数据怎么构造？有哪些来源？

A：五类来源，按质量排：

1. **人工标注**（最贵）：真实用户的问答、专家标注。
2. **强模型蒸馏**：用 GPT-4V/Claude 对图像生成多轮对话，再人工抽检。**注意只让它生成文本，不要用它生成图像**（版权与分布问题）。
3. **现有数据集改造**：VQA、检测（转成"图里有什么/在哪"）、OCR、grounding、文档理解，全部改写成对话格式。
4. **合成负样本**：故意问图中没有的东西，教模型说"没有"——**这是抑制幻觉最有效的数据手段**（对应 POPE 的 adversarial 设置）。
5. **纯文本数据混入**：防语言能力退化（见 A21）。

质量控制：去重（图像近重复是评测泄漏的头号来源）、答案长度控制（过长的回答会教模型"啰嗦"）、任务配比（OCR/推理/描述/grounding 要有比例）、难度分层。

A16 ★★ 怎么定图文 token 比例 / 数据配比？

A：没有普适公式，但有三条经验：

1. **按任务定 token 预算**：OCR/文档/grounding 需要高分辨率（给多 token），日常对话可以压缩（少 token）。
2. **按能力短板调**：评测发现哪块弱就加哪块数据（不是加权采样，而是补数据）。
3. **按阶段定比例**：对齐阶段图文对占绝对多数（可以 100%）；指令阶段通常图文 : 纯文本 ≈ 7:3 或 8:2。

监控手段：分别报"纯文本 benchmark"和"多模态 benchmark"，看是否出现"多模态涨、纯文本掉"的跷跷板。

A17 ★★ 分辨率策略有哪几种？各自代价？

A：四种：

策略	做法	token 代价	代表
固定分辨率	统一 resize 到 336/448	固定（576/1024）	LLaVA-1.5
动态切块（anyres）	大图切 N 块 + 1 张缩略图	N×576 + 576	LLaVA-1.6、InternVL
原生分辨率	按原分辨率切 patch，不缩放	与像素数成正比	NaViT、Fuyu
像素重排	2×2 patch 拼通道	÷4	InternVL

关键工程点：切块时块与块之间要**留重叠**（避免边界目标被切断），并做**位置编码的连续性处理**（否则模型不知道第 2 块在右边）。

A18 ★★ 多图 / 多轮对话在训练时怎么组织？

A：三条：

1. **特殊 token 标界**：每张图用 `<image>` / `<|image_pad|>` 包裹，模型靠这个知道"这里换了一张图"。
2. **图像只在首次出现时插入**：多轮里后续轮次引用"上面那张图"，不重复插 token（否则上下文爆炸）。
3. **loss mask**：只在 assistant 回复上算损失，用户提问和历史回复 mask 掉。

追问：多轮时历史图像 token 要不要丢？→ 上下文够就保留；不够时的常见策略是"旧图只留缩略图 token"或"旧图换成语义摘要"，本质是**压缩历史**。

A19 ★ 课程学习（curriculum）在 VLM 训练里怎么用？

A：由易到难排数据：短 caption → 长描述 → 单轮 QA → 多轮对话 → 需要推理/多图对比的难题。好处是**收敛更稳**（早期不被难题的噪声主导）、**避免早期过拟合到长答案格式**。

实践中最常见的"隐式课程"就是：对齐阶段全是短 caption（简单），指令阶段才上复杂任务。

A20 ★★ 合成数据有什么风险?

A：四个风险：

1. **分布坍缩**：全用强模型生成，学生会继承教师的偏好和错误（教师也可能幻觉）。
2. **风格单一**：句式、长度、格式趋同，模型失去多样性（用户会觉得"答得像机器人"）。
3. **事实污染**：教师对图像看错的地方，被当作"标准答案"教下去。
4. **评测污染**：合成时如果不小心用到评测集图像，指标虚高。

缓解：合成 + 人工抽检、保留一部分真实数据、拒绝采样（best-of-n 后人工/规则筛）、对合成数据降权。

A21 ★★ 混合模态训练时，怎么防止纯文本能力退化?

A：三条措施：

1. **数据层面保留纯文本**：图文:纯文本通常 7:3 ~ 8:2；SFT 阶段尤其重要。
2. **参数层面限制漂移**：用 LoRA 或只解冻部分层，减少对语言能力的破坏。
3. **评测层面同时盯**：每次报多模态分的同时报纯文本分，出现"跷跷板"就调整配比。

机制解释：图文数据的答案分布（描述性、短句）与文本数据（推理、长文）差异大，模型会向数据多的那侧偏——这叫**模态偏置**，不是简单的"遗忘"。

A22 ★★ 训练时视觉塔解冻有什么风险？什么时候该解冻?

A：**风险**：① 视觉塔参数量大，解冻后显存和算力翻倍；② 小数据下会破坏预训练特征（退化成"适配指令数据的私有特征"）；③ 与 LLM 的学习率不匹配时容易震荡。

该解冻的情况：领域差异极大（医学影像、红外、深度图、遥感）、需要极细粒度定位、数据量足够大（百万级以上）。**通常做法**：只解冻最后 1-2 层 + 小学习率（1/10），或加 LoRA。

A3 能力边界与失效模式（A23–A30）

A23 ★★★ VLM 的能力短板主要集中在哪五处?

A:

短板	表现	根因
细粒度计数	数错人数/物体数	对比学习不建模数量；token 聚合丢个体
空间关系	"左边的红球"搞错	全局对齐不建模相对位置；位置编码在视觉侧弱
精确 OCR	小字、密集文本读错	分辨率不足 + patch 太粗
视觉推理链	多步推理靠语言先验猜	训练数据里缺"看图推理"的中间步骤
grounding 精度	框偏、指代歧义	缺区域级监督；token 与像素的对应不唯一

面试加分：每说一个短板，配一句"所以工业上怎么绕"——如 OCR 靠 dynamic tiling + 专门 OCR 数据，空间关系靠显式 3D/2D 坐标输入（SpatialVLM 一类）。

A24 ★★★ 幻觉的三种成因在模型结构上分别对应什么?

A:

成因	结构层面的解释	缓解手段
语言先验主导	LLM 占了绝大部分参数，视觉 token 在序列里是"少数派"，attention 容易忽略它们	提高视觉 token 权重/数量、对比解码、加大视觉分辨率

成因	结构层面的解释	缓解手段
视觉信息不足	压缩过度（Q-Former/池化）、分辨率低 → 视觉证据本身弱	减少压缩、dynamic tiling、多图交叉验证
对齐错位	桥接层是"翻译器"，翻译错了 LLM 就会基于错误信息自信作答	对齐阶段加更多数据/多任务监督、区域级对齐

追问：为什么"加更多指令数据"不能根治幻觉？→ 指令数据教的是"怎么答"，不改变"看不看得清"；幻觉主要是**视觉证据强度**问题，得从分辨率/token/监督入手。

A25 ★★ 为什么 VLM 在计数上特别差?

A：四个原因叠加：

1. **训练目标不鼓励逐个体建模**：对比学习/生成目标都没有"必须数对"的压力。
2. **token 聚合丢失个体性**：patch 特征池化成全局语义时，5 个人和 6 个人的差异被平均掉。
3. **位置编码不显式表示"第几个"**。
4. **数据偏差**：训练数据里计数类问答少且答案分布偏（多半是"1 个""2 个"）。

能改善的做法：专门的计数数据 + 让模型输出检测框/点（"先定位再数"）+ 高分辨率。

A26 ★★ 空间关系理解为什么难?

A：因为 VLM 的语言侧对"左右上下"极敏感（文本里这两词很常见），但视觉侧**没有显式坐标系**——patch 序列的位置编码是 2D RoPE 或可学习编码，与"图像平面坐标系"不是一回事。于是模型倾向于**根据语言先验猜**（"盘子通常在下"），而不是看图。

工业解法：把坐标显式输入（"物体 A 在 (x1,y1)-(x2,y2)"）、加 3D/2D 位置监督数据、或干脆让模型先输出框再回答。

A27 ★★ OCR 能力怎么系统性提升?

A：四件事，按性价比排：

1. **分辨率**（最关键）：dynamic tiling 把长文档切成条带，每块单独编码。
2. **数据**：大量文档/表格/截图类 QA（DocVQA、ChartQA、TextVQA 一类），以及纯 OCR 转录任务。
3. **视觉塔**：patch 更小或专用 OCR 前端。
4. **后处理**：让模型输出结构化结果（JSON/表格），便于校验。

A28 ★★ VLM 做视觉 grounding 的三种输出形式?

A：① **文本坐标**（"`<box>(x1,y1,x2,y2)</box>`"）——最通用，但精度受 token 化影响（数字被切成多个 token）；② **特殊 token / 离散 bin**（把坐标量化成 bin 分类）——精度可控；③ **专用 head**（接一个检测头输出框/mask）——精度最高，但破坏了"纯语言模型"的统一性。

趋势：主流通用模型用 ①，专门的方向（指代分割、GUI agent）用 ②③。

A29 ★ 视频理解比图像多出来的难点是什么？（三条）

A：① **token 爆炸**（帧数 × 每帧 token）；② **时序建模**（帧间关系、事件先后、因果）；③ **长视频检索**（1 小时视频不可能全塞，必须"摘要 + 检索"）。附加难题：视频数据标注昂贵、评测集难做（时序问答的答案歧义大）。

A30 ★★ 为什么说"VLM 的数学推理很多时候是在做语言推理"?

A：因为它可以从题目文本推出答案（哪怕图没看对）。诊断方法很直接：**把图像遮挡/替换成无关图像，看答案是否改变**——不变就说明它没用图。这也是 HallusionBench 这类"视觉依赖测试"的设计思想。

A4 评测体系（A31–A38）

A31 ★★★ 主流评测集各自测什么？给一张能背的表。

评测集	测什么	需要记住的细节
MMBench	综合能力（20 个能力维度）	**CircularEval**：选项循环移位，全对才算对，消除位置偏置
MMMU	大学水平学科推理（图、表、公式、考试题）	多学科多图，考专业知识，人类专家也只有 ~80%
SEED-Bench	生成式理解（图+视频）	多项选择，人工校验过
POPE	**物体幻觉**	random / popular / **adversarial** 三设置；报 F1 和 yes 比例
HallusionBench	幻觉（视觉依赖 + 反事实）	专测"看图 vs 靠先验编"
MM-Vet	综合（识别+推理+OCR 交叉）	GPT-4 打分，看能力维度组合
RefCOCO/+/g	指代定位 grounding	Acc@0.5（IoU>0.5）或点命中率
TextVQA / DocVQA / ChartQA / OCRBench	OCR 与文档	工业落地最常看这四个
MathVista	数学视觉推理	图表+几何+公式
VideoMME / LongVideoBench	视频（短/中/长，有无字幕）	必须**分时长档**报，长视频普遍崩

A32 ★★★ CircularEval 是什么？为什么必须有？

A：把一道多选题的选项**循环移位 N 次**（N = 选项数），每次让模型作答，**N 次全对才算答对**。

原因：VLM 对选项位置有强烈偏置（训练数据里 A/B 出现频率高、或 token 化后某些位置的概率先验高），不移位会系统性高估能力——有研究报告这个偏差能贡献 10+ 个点的虚高。

面试意义：讲得出 CircularEval，等于告诉面试官"我读过评测实现，不只看排行榜数字"。

A33 ★★ POPE 的三个设置为什么要区分？

A：因为负样本的难度决定评测的可信度：

- **random**：随机挑不存在物体 → 太简单，模型靠"这词常见"就能答对；
- **popular**：挑数据集中高频但图里没有的物体 → 中等；
- **adversarial**：挑**与图中物体共现频率高**的物体（有椅子就问"有桌子吗"）→ 真考验"是否在看图"。

只看 random 分数会严重虚高，三个设置一起报才有意义。

A34 ★★ 评测里最容易自欺的四个地方？

A：① **数据泄漏**（训练图和评测图近重复，CLIP 系预训练数据几乎覆盖全网）；② **prompt 敏感**（换模板差几个点，不报 prompt 就是不完整）；③ **选项偏置**（不做 CircularEval）；④ **只报总分**（不报分项，OCR 强推理弱看不出来）。

加分补一句：还要报 **解码配置**（贪心/beam/温度）和**是否 few-shot**——这些不写，结果不可复现。

A35 ★★ 幻觉怎么量化？除了 POPE 还有什么办法？

A：四类方法：

1. **判别式探测**（POPE）：问"有没有 X"，看 yes/no 是否正确 → 简单、可自动化；
2. **生成式一致性**：同一张图问多次/换问法，答案不一致即为不稳定（self-consistency 视角）；
3. **视觉依赖测试**：把图替换成空白/无关图，答案不该变（该变才说明用了图）；
4. **人工/强模型打分**：MM-Vet 式，成本高但是最终裁判。

A36 ★★ grounding 的评测为什么比检测的 mAP 更"宽容"?

A：因为 grounding 的目标是"**指代唯一性**"而不是"检测全面性"：输入是一句描述，只要求找到**那个**目标，不惩罚漏检其他物体。指标常用 Acc@0.5 或点命中（点落在 GT mask 内），后者更宽容（不要求框贴合）。

难点在指代歧义：图里有三只狗，"那只左边的狗"需要理解关系，而不是靠类别先验。

A37 ★ 长视频评测为什么必须分档报?

A：短视频（<2 分钟）靠帧采样就能覆盖，模型表现接近图像；中长视频开始依赖时序推理与检索，性能断崖式下降。**只报平均分会掩盖这个断崖**——所以 VideoMME 明确分短/中/长三档，并区分有无字幕（有字幕时模型可能"读字幕"而不看画面，这也是要看分项的原因）。

A38 ★★ 怎么评测"模型有没有真在看图"？设计一个实验。

A：三个可落地实验：

1. **图像替换**：换成同类无关图 → 答案应显著改变（不变 = 没用图）。
2. **局部遮挡**：遮住关键区域 → 答案应变差（不变 = 没依赖该区域）。
3. **反事实图像**：把"红色苹果"改成蓝色 → 答案应跟着变（不变 = 靠先验）。

这三个也是 HallusionBench 的核心思想。**面试时能讲出实验设计，比背分数有用。**

A5 推理与部署（A39–A45）

A39 ★★★ VLM 推理的延迟构成？哪一段是瓶颈?

A：四段：① **预处理**（resize/切块/归一化，CPU 上常被忽略但很贵）；② **视觉塔前向**（与分辨率平方相关）；③ **prefill**（把图文 token 一次前向，计算密集）；④ **decode**（逐 token，访存密集，与输出长度线性）。

瓶颈判断： - 高分辨率 + 短回答 → 卡在 ②③（视觉塔 + prefill）； - 低分辨率 + 长回答 → 卡在 ④（decode，KV cache 带宽）。

这句话很值分："端侧 VLM 要先分清是 prefill-bound 还是 decode-bound，两者的优化手段完全不同。"

A40 ★★ VLM 的 KV cache 和纯文本 LLM 有什么不同?

A：机制一样（缓存每层 K/V），但 VLM 有两处特殊：

1. **视觉 token 的 KV 是固定的**：同一张图多轮问答时，视觉部分的 KV 可以**跨轮复用**（prefix cache 思路）——这是多模态特有的优化收益。
2. **视觉 token 数量大且不等于文本长度**：一张 anyres 图可能 2880 token，KV cache 直接翻几倍，长多轮时显存爆得比纯文本快得多。

A41 ★★ 图像 token 太多，有哪些工程压缩手段?

A：五类：① 视觉侧压缩（pixel shuffle、池化）；② 桥接层压缩（Q-Former/resampler）；③ token 剪枝（按注意力重要性保留 top-k）；④ 动态分辨率（按任务给 token）；⑤ 缓存复用（同一图的多轮只算一次）。

面试提醒：压缩一定损精度，所以要说清"在哪一层压、损失什么"——通道压缩损细节，序列压缩损定位，两者可叠加但要注意叠加放大误差。

A42 ★★ VLM 量化有什么特别的坑?

A：三个：① **视觉塔对量化更敏感**（输入分布极端、动态范围大），常见做法是视觉塔保 FP16、只量化 LLM；② **视觉 token 的离群值**（ViT 输出常有较大激活）会破坏 INT8，需要 per-channel 或对视觉通路单独处理（类似 AWQ 保护重要通道）；③ **KV cache 量化**在多轮/长上下文里收益最大，但要单独校准。

A43 ★ 投机解码能用在 VLM 上吗?

A：能，但草稿模型的输出分布要覆盖**图文场景**——用纯文本小模型做草稿，遇到需要"看细节"的问题命中率会掉。实践中：① 用同系列的小 VLM 作草稿；② 或者只在"回答格式固定/重复性高"的场景（如结构化抽取）用。

A44 ★★ 端侧 VLM 的现实约束是什么?

A：四条硬约束：① **内存**（权重 + KV cache + 激活，手机级别只有几 GB 可用）；② **算子支持**（动态形状、动态分辨率、attention 变体在 NPU 上常常要改）；③ **功耗/发热**（长时间推理会降频）；④ **预处理开销**（切块/resize 在 CPU 上可能比模型还慢）。

你的接口（可直接讲）："我在 MDC610 上做过 ONNX→ATC/AOE 的完整链路，所以端侧 VLM 我会先问三件事——**目标 SoC 的算子白名单、内存预算、是否允许动态分辨率**。"

A45 ★ 昇腾/国产芯片上部署 VLM，主要工作量在哪?

A：四块：① **算子适配**（不支持的算子拆解或等价重写，例如动态 attention、top-k、动态 shape）；② **shape 固定化**（把动态分辨率按档位枚举，分桶推理）；③ **精度对齐**（fp16 溢出、算子融合顺序导致的数值差异，要逐层比对）；④ **内存规划**（模型段/工作空间手工分配，KVCache 动态增长需要预分配）。

A6 VLM 的损失函数与评测指标（A46–A57）

> 一句话前提：**VLM 的损失 90% 就是"文本侧的下一个 token 交叉熵"**，其他都是辅助。指标则完全分任务走。

A46 ★★★ VLM 训练里最主要的损失是什么？为什么不是 MSE?

A：**自回归的 token 级交叉熵（next-token CE / NLL）**：

```
L = −(1/T) Σ_t log p_θ( y_t | y_<t , 视觉token , 指令 )
```

为什么不用 MSE： 1. 语言是**离散分布**，MSE 假设高斯连续误差，与概率建模不匹配（见 B13/B97）； 2. CE 对"自信但错"的惩罚趋于无穷，天然抑制过度自信； 3. 生成任务的目标是**分布匹配**（每个位置的概率分布），不是回归某个数。

注意：图像**生成**（扩散）才用 MSE（噪声预测），这与 VLM 的文本生成是两套目标（A55）。

A47 ★★★ 对齐阶段（stage-1）常见的损失组合有哪些?

A：四种流派，取决于模型结构：

流派	损失组合	代表
生成式对齐	next-token CE（冻结 LLM，只训 projector）	LLaVA 两阶段的第一阶段
对比 + 匹配 + 生成	ITC（InfoNCE）+ ITM（BCE）+ ITG（CE/MLM）	BLIP、ALBEF
纯对比（双塔）	对称 InfoNCE（或 SigLIP 的 pairwise sigmoid）	CLIP、ALIGN、SigLIP
多任务 query 对齐	ITC + ITM + ITG，Q-Former 输出接冻结 LLM 做 LM loss	BLIP-2 两阶段

关键差别：**对比损失（ITC）学"全局是否匹配"，生成损失（CE）学"细节怎么描述"**。BLIP/ALBEF 两者都要，是为了兼得；LLaVA 只保留生成，是因为它有海量指令数据 + 强视觉塔，不需要靠对比来逼对齐。

A48 ★★ 为什么 ITC 用 InfoNCE 而不是 MSE?

A：三层原因：

1. **目标不同**：ITC 要的是"**相对排序**"——同一 batch 里，正确配对的相似度要高于其他配对。InfoNCE 是 softmax 分类（正确配对是标签），天然表达"区分度"；MSE 只要求"数值接近"，学不出"相对更近"。
2. **梯度行为不同**：InfoNCE 的梯度是 `p − y`，在"把错误配对排到前面"时梯度大；MSE 在大 batch 的 B×B 矩阵上会被大量"本来就对的负样本对"稀释。

3. **信息论视角**：InfoNCE 是**互信息的下界**（这也是"多模态对齐 = 最大化互信息"这个说法的来源）。

追问：负样本从哪来？→ **in-batch negatives**（batch 内其他样本的图/文），所以大 batch 是 CLIP 的命门；SigLIP 改成 pairwise sigmoid 后就不再需要全 batch 归一化。

A49 ★★ ITM（图文匹配）为什么必须有负样本？怎么挖 hard negative？

A：ITM 是二分类（匹配/不匹配），**没有负样本就无法训练**。

hard negative 的挖法（ALBEF/BLIP 的做法）： - 用 ITC 算 batch 内相似度矩阵，**取"相似度高但不是正确配对"的样本**作为 hard negative（如把 A 的图配 B 的相似文本）； - 这些样本在语义上接近、但确实不匹配，最能教会模型细粒度区分。

为什么需要 hard negative：随机负样本太容易（"猫的图 + 汽车的描述"），模型学不到细粒度；只有"猫的图 + 狗的相似描述"才能逼模型去看细节。

A50 ★★★ SFT 阶段的 loss mask 怎么做？为什么只对 assistant 算？

A：做法：把序列中 user 的提问、system prompt、以及（多轮里的）历史 assistant 回复都贴 label = `-100`（PyTorch 的 `ignore_index`），只有**当前轮的 assistant 回复参与 loss**。

三个原因： 1. **目标任务明确**：我们训练的是"看到指令后如何回答"，不是"复述指令"。 2. **避免模型学会"自问自答"**：如果对 user 也算 loss，模型会倾向于生成用户的问题（尤其在多轮里出现"接着提问"的退化）。 3. **梯度更集中**：user 部分通常比回答长（尤其检索增强的长上下文），不过滤会浪费大量算力。

工程细节：多轮对话里，历史 assistant 回答**通常也 mask 掉**（只算最后一轮），否则模型会被"重复自己说过的话"带偏；但有些实现会算全部轮次（相当于更多训练信号），两种都有人用——面试时能说出这个取舍很加分。

A51 ★★ RLHF 的两个损失分别是什么？

A：两段：

① 奖励模型的 Bradley-Terry 损失（给偏好对 chosen / rejected）：

```
L_RM = −log σ( r(x, y_w) − r(x, y_l) )
```

② PPO 的 clip 目标（策略优化）：

```
ratio = π_θ(a|s) / π_old(a|s)
L_clip = −E[ min( ratio · A ,  clip(ratio, 1−ε, 1+ε) · A ) ]
总奖励 R = r_RM − β · KL(π_θ ‖ π_ref)
```

三个关键设计： - **clip** 限制单步更新幅度（防止策略崩）； - **KL 惩罚** 防止模型为了拿高分而跑偏成"奇怪的表达"； - **GAE**（广义优势估计）把稀疏奖励分摊到每个 token。

多模态的特殊性：RM 本身要是个 VLM（得看图），所以 RLHF 在多模态里成本约为纯文本的 2~3 倍。

A52 ★★ DPO 损失是怎么来的？实现要点？

A：DPO 把"RM + RL"合并成一个分类式损失，**直接用偏好对优化策略**：

```
L_DPO = −log σ( β · [ log(π_θ(y_w|x)/π_ref(y_w|x)) − log(π_θ(y_l|x)/π_ref(y_l|x)) ] )
```

推导要点（能说清这段很值钱）：在 KL 约束下最优策略有闭式解 `π* ∝ π_ref · exp(r/β)`，反解出 `r = β · log(π*/π_ref) + const`；把它代回 Bradley-Terry 损失，就得到上式——**奖励被"吸收"进策略里了**，所以不需要单独训 RM。

实现要点： - 需要**两个前向**（policy 和 ref）算 4 个 logprob（chosen/rejected × policy/ref）； - `β` 常取 0.1~0.5，越大越保守； - 用 **logprob 之和**而不是平均（长度归一化方式会影响效果，有论文专门讨论）。

与 PPO 的取舍：DPO 便宜稳定但离线（无探索），PPO 能在线探索但工程复杂、需要 4 个模型。

A53 ★ GRPO 与 PPO 在损失层面的差别？

A：GRPO **去掉了 value network（critic）**：对同一个 prompt 采样一组（G 个）输出，用**组内相对优势**当基线：

```
A_i = (r_i − mean(r_1..r_G)) / std(r_1..r_G)
L = − E[ min(ratio · A_i, clip(ratio,1−ε,1+ε) · A_i) ] + β · KL
```

好处：省掉 critic 网络（显存和调参都省一大块），且组内归一化自带方差缩减。**代价**：需要同一 prompt 多次采样（推理成本上升），组太小或奖励方差为零时信号很弱。

A54 ★ 带任务头时（grounding / 分割 / OCR）损失怎么加?

A：分两种情况：

① 纯文本输出坐标（主流通用 VLM）：把坐标当字符串生成 → **还是 next-token CE**，但要注意坐标数值被 tokenizer 切成多个 token，导致精度受限（见 A28）。可以加**坐标回归辅助头**补精度。

② 专用头（分割/检测）：

任务	损失	注意
检测/grounding	分类 focal + 框 SmoothL1 + GIoU/DIoU	与 S9 一致；框回归要归一化
指代分割	BCE（主）+ Dice（辅）	正样本极少，用 focal 或权重
OCR / 转录	token 级 CE	长文本要注意长度归一化
密集预测（深度/占用）	任务专用（见 S9/C8）	与文本损失量纲差几个数量级**必须归一化**

关键工程点：文本 CE 的量级通常是 1~3，而框回归/深度损失常是 0.01~0.1，**直接相加会让文本损失主导**——必须做量纲归一化（S9 Q111）。

A55 ★ 图像生成侧的损失和 VLM 的文本损失有什么本质区别?

A:

	VLM 文本生成	扩散图像生成
目标	离散 token 的**概率分布**	连续噪声的**回归**
损失	交叉熵（逐 token）	**MSE**（预测噪声 ε 或 v 或速度场）
输出	每步一个 token 的分类分布	每步一个连续张量
采样	逐 token 自回归	多步去噪（DDIM 20~50 步）

统一视角：两者都是"从噪声/前缀恢复数据"，但 VLM 在**离散序列空间**做分类，扩散在**连续空间**做回归。新一代"统一模型"（Chameleon/Emu3 用离散 token、Transfusion 用双头）就是在试图把这两套目标合到一起——这也是当前的开放问题（A53）。

A56 ★★★ VLM 的评测指标分哪几类？各举必背的

A:

类别	指标	说明
综合理解	MMBench / MMMU / MM-Vet / SEED 的**准确率**	多为多项选择 → acc；MM-Vet 用 GPT-4 打分
VQA	VQA accuracy（`min(#同答/3, 1)`）	VQA v2 的老口径，10 个标注者投票
幻觉	POPE 的 **Accuracy / F1 / yes 比例**；HallusionBench 的 acc	见 A33
定位	RefCOCO 的 **Acc@0.5** 或点命中率	见 A36
OCR/文档	**ANLS**（DocVQA）、精确匹配（OCRBench）	ANLS 对编辑距离做归一化，比精确匹配宽容
生成（描述）	CIDEr / BLEU / METEOR	与人类判断相关性弱，现在多用 GPT 打分
生成（图像）	FID / IS / CLIPScore	见 C49
检索	Recall@K / mAP	双塔模型的看家指标
效率	TTFT（首 token 延迟）、TPS（每秒 token）、峰值显存	落地必看

必背的三条"口径提醒"：① MMBench 必须 CircularEval；② POPE 必须报三设置的 adversarial；③ 生成类指标（CIDEr/FID）与人类判断弱相关。

A57 ★★ 为什么"损失降、指标不涨"在 VLM 上特别常见?

A：四个 VLM 特有的原因：

1. **损失是 token 级，指标是回答级**：一个回答 50 个 token，错 1 个 token（比如数字写错）loss 只升一点，但整个答案判错 → **准确率是"全对才算对"，比 loss 严格得多**。
2. **格式比内容更容易被 loss 奖励**：模型学会"先复述问题再答""用 markdown 列表"这类高概率模式就能显著降 loss，但指标不变。
3. **训练分布与评测分布不同**：SFT 数据的问法、长度、语言与评测集差异大（评测集多是短问答 + 多选）。
4. **有截断/格式不匹配**：评测要抽取答案，格式不对就判错（明明答对了）。

诊断手段：把 loss 拆成"格式 token（标点、连接词）"和"内容 token（实体、数字）"分别看——如果降的是格式 token，那指标不涨就是合理的。**这个做法很实用，面试可以直接讲。**

A7 Encoder / Decoder 的辨析（A58–A65）

> 这个词在一句话里可能有三种含义，**先问清是哪一种**，否则答得越多越偏。

A58 ★★★ 先分清三种 "encode/decode"

A：

语境	encode	decode	例子
Transformer 架构	双向注意力，输出表征	因果注意力，输出下一 token	BERT vs GPT
VLM 角色	视觉编码器 / 文本编码器（理解）	语言模型（生成）	ViT + LLaMA
工程压缩	把数据压成潜变量	把潜变量还原	VAE encoder/decoder、视频编解码
序列化	字符串 → token id	token id → 字符串	tokenizer

面试时第一句就说："您指的是 Transformer 的 encoder/decoder，还是 VLM 里视觉编码器 vs 语言模型？"——**这一问就能避免 80% 的误解**。

A59 ★★★ Transformer 里 encoder 和 decoder 的本质区别是什么?

A：**一处核心差异 + 三处衍生**：

```
核心：注意力 mask
   Encoder：全注意力（每个位置看所有位置）          → 双向上下文
   Decoder：因果 mask（位置 i 只看 ≤ i）           → 单向上下文（自回归）
```

	Encoder	Decoder
注意力	双向（无 mask）	因果 mask（下三角）
输出用途	**表征**（给下游分类/检索/匹配）	**下一 token 的分布**（生成）
训练目标	MLM / 对比 / 回归	next-token CE
训练并行	可并行	**也可并行**（见 A60）
推理	一次前向	必须串行（每步依赖前一步输出）

一个常考细节：encoder 里通常**没有** cross-attention，decoder 里**可以有** cross-attention（看 encoder 的输出，如 T5/BART）；decoder-only 模型（GPT/LLaMA）则完全没有 cross-attention 层。

A60 ★★★ 为什么 decoder 训练能并行、推理必须串行?

A：

训练能并行：靠 **teacher forcing + causal mask**。把完整目标序列一次性喂进去，mask 保证位置 i 只能看到 ≤ i 的真实 token，所以**所有位置的预测可以同时算**（一次前向得到 T 个位置的 loss）。

推理必须串行：因为位置 t 的输入是模型自己刚生成的 t−1 个 token，**没有 GT 可喂**，只能逐步生成。

KV cache 的意义：串行时每步都要重算全部历史，代价 $O(L^2)$；缓存历史 K/V 后每步只算 1 个新 token 的 K/V + 与全部历史的注意力，**每步 O(L)**，总 $O(L^2)$ 但常数小得多。

一句话："训练时把已知答案当输入（可以并行），推理时答案要自己产生（只能串行）——**这就是为什么大模型推理是 memory-bound**。"

A61 ★★ encoder-decoder 架构（T5 / BART）和 decoder-only 有什么不同?

A：

	encoder-decoder	decoder-only
结构	encoder（双向）+ decoder（因果 + cross-attention）	单塔因果
典型	T5、BART、Whisper、部分多模态（BLIP）	GPT、LLaMA、LLaVA
优势	输入理解更深（双向）；输入输出分离清晰	结构简单、权重可复用、few-shot 能力好
劣势	参数与工程更复杂；难做"输入输出同分布"的任务	输入也走单向，理论上理解力弱一点

关键差别在"输入能否双向编码"：encoder-decoder 让输入被完整双向理解后再生成；decoder-only 的输入 token 也只能看到自己之前的 token（虽然对生成式任务影响不大，但对"精确检索长上下文"这类任务理论上吃亏）。

A62 ★★★ 在 VLM 里谁是 encoder、谁是 decoder?

A：

```
视觉侧：ViT/SigLIP  →  encoder（双向，输出 patch token）
桥接：  projector / Q-Former → 既不是典型 encoder 也不是 decoder（对齐模块）
语言侧：LLM → 主流是 decoder-only（因果）
```

所以一个典型 VLM（LLaVA 类）是 **"encoder(视觉) + decoder-only(语言)"** 的异构组合，**不是 encoder-decoder 架构**。这一点很多人会说错。

例外：BLIP 用的是 encoder-decoder（BERT 式文本编码器 + 解码器）；CLIP 是**双 encoder**（两个都是 encoder，都没有 decoder）。

A63 ★★★ 为什么主流 VLM 选 decoder-only?

A：五个理由：

1. **生态与权重复用**：直接拿现成 LLM（LLaMA/Qwen）当底座，省掉从零训。
2. **任务统一**：所有任务都变成"下一个 token 预测"——视觉问答、描述、grounding（输出坐标字符串）都是同一个接口，不需要为每个任务设计 head 和 loss。
3. **数据效率**：指令数据天然是"对话续写"的形式，decoder-only 的 SFT 非常直接（A50 的 mask 策略）。
4. **few-shot / in-context 能力强**：这是 decoder-only 的固有优势（自回归建模带来的分布内插）。
5. **推理框架成熟**：KV cache、continuous batching、投机解码都是围绕 decoder-only 成熟的。

代价（见 A64）：视觉与文本的交互是**单向**的（视觉 token 只被文本"看到"，视觉不反向查询文本）；另外多图/长文档时，输入侧也只能单向处理。

A64 ★★ decoder-only 做 VLM 有什么代价?

A：四条：

1. **单向融合**：视觉 token 无法"回看"文本（不像 cross-attention 可以让视觉查询文本），所以**细粒度指代**（"第三个按钮"）依赖 LLM 自己学会用位置信息。
2. **输入不能双向编码**：长文档/多图场景下，前面的信息只能通过单向注意力传下去。

3. **token 预算紧张**：视觉 token 和文本 token 共享同一个上下文窗口（decoder-only 没有独立的 encoder 侧），所以 token 压缩格外重要。
4. **推理延迟**：图像 token 全在 prefill 里，长图会显著抬高 TTFT（首 token 延迟）。

取舍结论：这些代价被"生态 + 数据效率"的优势覆盖，所以工业上**几乎没有例外地选 decoder-only**；需要双向融合时，用 **Q-Former / 双向编码的视觉塔 / 专门的 cross-attention 层（Flamingo）**来补。

A65 ★ encoder 的双向注意力对多模态融合有什么优势？（开放题）

A：三条优势 + 一条代价：

优势： 1. **区域与词的细粒度对应更强**：双向注意力让每个图像 patch 都能"看到"全部文本，从而学到"这个词对应这块区域"——这是 ALBEF/BLIP 里 ITM 头的能力来源。 2. **检索/匹配更准**：双向编码的融合表征适合"判断是否匹配"，比单向拼接更强。 3. **适合做** grounding/指代这类"输入侧就要求对齐"的任务。

代价：不能直接生成自由文本（需要解码器）。

所以在工程上的常见组合是：双向 encoder 做融合/对齐（用于检索、匹配、定位），decoder-only 做生成（用于问答、对话）——这也是"双塔 + 生成塔"混合模型（如 CoCa 的对比 + caption 双头）的思路。

A8 LoRA 与 PEFT 微调（A66–A77）

> LoRA 是大模型微调的**事实标准**，也是面试的高频点。这一节把"为什么有效、省了什么、怎么选参、怎么避坑"讲透，并给出一套能直接报出来的数字。

A66 ★★★ LoRA 的数学形式是什么？为什么它有效？

A：冻结原权重 $W \in R^{d_{out} \times d_{in}}$，只学一个低秩增量：

$$W' = W + \Delta W = W + B \cdot A \qquad A \in R^{r \times d_{in}},\ B \in R^{d_{out} \times r},\ r \ll \min(d_{in}, d_{out})$$

$$\text{前向：}\ h = Wx + (\alpha/r) \cdot B \cdot A \cdot x$$

有效的原因（三层）： 1. **微调的增量本身就是低秩的**：已有研究（Aghajanyan 等）表明，预训练模型适配下游任务所需的"内在维度"远小于参数量——**ΔW 的秩很低**，所以低秩近似几乎不损失信息。 2. **不破坏预训练知识**：W 完全冻结，只是加一个"旁路修正"，天然避免了灾难性遗忘。 3. **优化更简单**：可训练参数少 2~3 个数量级，优化器状态和梯度都小，等效于强正则。

一句话："LoRA 不是压缩，而是**假设微调只需要在少数方向上修正**。"

A67 ★★★ A 和 B 为什么要这样初始化？α/r 是干什么的？

A：

初始化：**A 用高斯随机、B 全零** → 训练开始时 $\Delta W = B \cdot A = 0$，模型的输出与原始模型**完全一致**。

为什么必须这样：如果 A、B 都随机，初始的 ΔW 就是一个随机扰动，相当于给已经训好的模型"注入噪声"，训练早期会出现 loss 尖峰甚至破坏原有能力。**这和我们做多任务时用"零初始化残余分支"保安全的思路是同一件事**（恒等初始化 → 开新分支不影响原行为）。

α/r 的作用：缩放因子。`alpha` 是"学习率的另一种表达"——

$$h = Wx + (\alpha/r) \cdot BAx$$

- 固定 α 而增大 r 时，α/r 变小，抵消了"参数变多导致的有效步长变大"，**所以改 r 时不必大改学习率**；
- 实践惯例：**α = r 或 α = 2r**（如 r=16、α=32）；rsLoRA 提出改用 $\alpha/\sqrt{r}$，让不同 rank 下的更新尺度更一致。

附：r 与 α 到底各管什么（实测验证）

分工一句话：r 管"能改多少"（容量），α 管"改得多用力"（幅度），两者一起决定有效更新量。

① r = 秩 = 容量的上限（$\Delta W = BA$ 的秩最多是 r）

r	参数量 r(d_in+d_out)（d=256 示例）	相对原始权重	实测 rank(BA)
1	512	0.78%	1
4	2,048	3.12%	4
16	8,192	12.50%	**16**
64	32,768	50.00%	64

含义：**r 就是"允许模型在多少个方向上修正原权重"**。r 太小 → 装不下任务需要的改动（欠拟合）；r 太大 → 参数量涨、过拟合风险上升、adapter 文件变大。

② ΔW 的幅度随 $\sqrt{r}$ 增长（这才是 α 存在的理由）

r	‖BA‖_F	‖BA‖_F / $\sqrt{r}$	乘 α/r（α=32）后	乘 α/$\sqrt{r}$ 后
4	0.0510	0.0255	0.4080	0.8160
16	0.1026	0.0257	0.2053	0.8211
64	0.2050	0.0256	0.1025	0.8200
256	0.4092	0.0256	0.0512	0.8185

读三件事： 1. ‖BA‖_F / $\sqrt{r}$ 几乎恒定（0.0256）→ **同样初始化下，r 越大 LoRA 分支的输出越"响"**（幅度 ∝ $\sqrt{r}$）； 2. 用 **α/r** 缩放时，r 越大有效幅度越小（∝ 1/$\sqrt{r}$）→ **大 rank 会学不动**，这是 α/r 的缺陷； 3. 用 **α/$\sqrt{r}$** 缩放时幅度与 r 无关（0.82 恒定）→ **这正是 rsLoRA 的动机**。

③ r=16、α=32 具体意味着什么

```
α/r = 32/16 = 2      → LoRA 分支的输出被放大 2 倍后加到原输出上
可训练参数（7B、全线性层）= 39.98M = 0.57% 的模型参数
adapter 文件（fp16）≈ 80 MB
```

- 缩放 = 2 是"略微主动"的设置（α=r 时缩放为 1，更保守；α=4r 则更激进）；
- 这个组合是社区最常见的默认值之一（LLaMA-Factory / PEFT 示例里大量出现），**但它不是普适最优**——任务差异大时该往上调 rank。

④ 三条实践规则 1. **改 r 就按比例改 α**（保持 α/r 不变），否则学习率的实际效果会意外变化； 2. **α 太小**（缩放 < 0.5）→ 学不动、loss 平；**α 太大**（缩放 > 4）→ 训练初期输出被扰动，loss 尖峰甚至破坏原模型（本质是"学习率太大"）； 3. rank 开到 **64 以上**时，改用 α/$\sqrt{r}$（rsLoRA 式）避免信号衰减。

A68 ★★★ LoRA 到底省了什么显存？（算一遍账）

A：以 **LLaMA-7B** 为例（hidden 4096、FFN 11008、32 层）：

配置	可训练参数	占 7B	LoRA 侧显存（bf16 参数+梯度+Adam）
全参微调	7B	100%	权重 14GB + 梯度 14GB + 优化器 **84GB** = **约 112GB**（不含激活）
LoRA r=8，全部线性层	19.99M	**0.29%**	约 0.28 GB
LoRA r=16，全部线性层	39.98M	0.57%	约 0.56 GB
LoRA r=32，全部线性层	79.95M	1.14%	约 1.12 GB
LoRA r=16，**仅 q/v**	12.58M	0.18%	约 0.18 GB

关键认知（面试常被追问）： - LoRA 省掉的是 **① 梯度 ② 优化器状态（Adam 的 m/v/主权重 = 12 字节/参数）**； - **不省** ① 冻结权重本身（7B 的 bf16 仍占 14GB）、② **激活**（随 batch×序列长度增长，长上下文仍是 OOM 主因）。 - 所以"LoRA 之后还 OOM → 该上梯度检查点 / FlashAttention / 减小序列，而不是调 rank"。

adapter 文件大小：r=16 全线性层约 **80MB**（fp16），这就是你能随手分享、多任务切换的物理基础。

A69 ★★★ rank 和 target modules 怎么选?

A:

rank: - 起步 **8~16**，实测不够再翻倍（16→32→64）； - **判断够不够**：训 loss 与目标任务指标在更大 rank 下**不再改善** = 已饱和； - 规律：**任务越"远离"预训练分布（新语言、新领域、代码/数学），需要越大 rank**；纯格式/风格适配用小 rank 即可。

target modules（加到哪些层）： - **最小可用**：`q_proj, v_proj`（原论文的做法）； - **推荐**：`q, k, v, o` + **MLP 的 gate/up/down**（全线性层）——实测通常比只加 q/v 好 1~3 个点，参数只多 3 倍（相对 7B 仍是 0.5% 量级）； - **注意**：不同框架的层名不一样（LLaMA 系是 `q_proj/k_proj/v_proj/o_proj/gate_proj/up_proj/down_proj`，Qwen 系可能是 `c_attn/w1/w2`），配错名字会静默地"什么都没训"——**必须打印可训练参数数量校验**。

A70 ★★ 训练完怎么合并？有哪些坑?

A:

```
推理前合并：  W_merged = W + (α/r)·B·A      ← 合并后无任何额外延迟
不合并（在线）：h = Wx + (α/r)·BAx          ← 多一次小矩阵乘，约 1~3% 延迟
```

四个坑： 1. **精度损失**：在 fp16 下 $W + \Delta W$ 会因舍入丢信息，尤其 ΔW 很小时。**做法：把 W 升到 fp32 合并后再转回 bf16/fp16**。 2. **合并后不可逆**：想换 adapter 就得保留原始权重（PEFT 的做法是保存 adapter + 基座指针）。 3. **多 adapter 不能直接相加**：两个在不同任务上训的 ΔW 相加，会得到"什么都会一点、什么都不精"的结果；正确做法是**切换**（加载不同 adapter）或重新联合训练。 4. **量化基座上的合并**：QLoRA 训完若直接合回 4bit，精度掉得厉害；**标准做法是先在 fp16 合并，再重新量化**（或干脆不合并、推理时动态加）。

A71 ★★★ QLoRA 的三个关键技术是什么?

A:

1. **NF4（4-bit NormalFloat）**：一种**按分位数**设计的 4bit 数据类型，对"近似正态分布"的权重是最优的信息论编码——比普通 INT4 精度更高。
2. **双重量化（Double Quantization）**：量化本身需要"量化常数"（每个 block 一个 scale），这些常数也占空间；**把量化常数再量化一次**，平均每参数省约 0.37 bit（65B 模型约省 3GB）。
3. **分页优化器（Paged Optimizers）**：用 NVIDIA 的统一内存，把优化器状态在显存/内存间按页换入换出，**避免长序列时突然 OOM**。

效果：论文在**单张 48GB GPU 上微调 65B 模型**。7B 的 4bit 基座约 **3.5GB**，加 LoRA 与激活，单张 24GB 消费级卡就能训。

代价：① 训练速度变慢（反量化开销）；② 4bit 有精度损失（通常比 LoRA 略差）；③ 合并麻烦（见 A70 第 4 条）。

A72 ★★ DoRA / rsLoRA / LoRA+ 各改了什么?

A:

变体	改了什么	直觉
DoRA（Weight-Decomposed）	把权重分解成 **幅度 m × 方向 V/‖V‖**，LoRA 只训方向、幅度单独训	微调主要改"方向"，改幅度反而破坏知识 → 小 rank 下明显更稳、更准
rsLoRA（rank-stabilized）	缩放改成 **α/√r**	大 rank 时 α/r 衰减太快导致学不动，$\sqrt{r}$ 让不同 rank 的更新尺度一致
LoRA+	**A 和 B 用不同学习率**（B 用大 lr，如 16×）	B 初始为零、A 是随机，两者梯度量级不同，同步 lr 会浪费一半
AdaLoRA	用 SVD 形式**自适应分配 rank**（重要的层给大 rank）	层间重要性不同
PiSSA	用**权重的 SVD 主成分**初始化 A/B	从"最重要的方向"开始学，收敛更快

面试建议：默认用 LoRA；小 rank + 要更好效果时试 DoRA；rank 开大（>64）时用 rsLoRA。

A73 ★★ LoRA 和其他 PEFT 方法怎么对比?

A:

方法	做法	推理延迟	效果	备注
LoRA	低秩旁路 ΔW	**可合并为零**	强	主流
Adapter	插入小 MLP 模块（串行）	**有**（除非剪枝）	强	破坏并行、层数变深
Prefix / P-Tuning	学一段虚拟 token 前缀	占上下文	中（对生成任务偏弱）	省参数但不是"改权重"
IA3	学三个缩放向量（k/v/FFN）	可合并	中	参数极少（万级）
BitFit	只训 bias	零	中偏弱	极简基线
全参微调	训全部权重	零	**上限最高**	显存 ~16× 参数

一句话："LoRA 赢在**无推理延迟 + 效果接近全参 + adapter 可切换**这三点的组合，其他方法都缺一项。"

A74 ★★★ VLM 微调时 LoRA 加在哪里?

A：三个部位，优先级不同：

部位	是否加 LoRA	理由
projector / 桥接层	**通常全参训练**（不加 LoRA）	它本来就是新加的、参数量只有几 M，加 LoRA 反而限制了学习
LLM	**加 LoRA**（q/k/v/o + MLP）	主力，参数量占 99%
视觉塔	**默认冻结**；领域差异大时加 LoRA（只加最后几层）	见 A22；加 LoRA 在视觉塔上时通常用更小的 rank（4~8）

典型配置（可直接讲）：

```
视觉塔：冻结（或最后 2 个 block 加 rank=4 的 LoRA）
projector：全参
LLM：LoRA r=16, alpha=32, target=q,k,v,o,gate,up,down
学习率：LLM 的 LoRA 用 1e-4 ~ 2e-4（比全参微调大 10 倍，因为可训参数少）
```

一个坑：VLM 里如果只给 LLM 加 LoRA、projector 也冻结，模型几乎学不动（projector 是随机初始化的话更糟）—— **projector 必须可训**。

A75 ★★ 多任务多 adapter 怎么管理？切换代价是什么?

A:

方式： 1. **每个任务一个 adapter**（共享同一基座）：存储省（每个几十 MB）、切换快； 2. **一个 adapter 混训多任务**：更省显存、但任务间会互相干扰（小 rank 下尤其明显）； 3. **MoE-LoRA**（每个任务一个 LoRA 专家 + 路由）：灵活但要额外训练路由。

切换代价：加载 adapter 需要把 BA 加进（或替换掉）当前权重——**如果合并过就必须重载基座**（代价大）；**如果保持分离，只需换 adapter 指针**（毫秒级，但推理时有小量额外算力）。

实践建议：训练时保持分离（方便切换），部署时对每个任务分别合并（拿零延迟）。

A76 ★★ LoRA 微调的五个常见坑

A:

1. **target modules 名字写错** → 静默地什么都没训（loss 看起来还在降，因为训的是 projector/bias）。**必须打印可训练参数数量**。
2. **rank 太小却指望大改能力**（比如给中文模型加英文任务）→ 欠拟合，表现为"loss 降不下去、输出风格不变"。
3. **改了 r 没改 α** → 有效学习率意外变化（见 A67）。
4. **合并精度损失** → fp16 下合并后效果比"不合并"差；用 fp32 合并。

5. **以为 LoRA 能解决长上下文 OOM** → LoRA 不省激活，长序列仍要靠梯度检查点/FlashAttention。

一个隐蔽的坑：`eval` 时忘了启用 adapter（PEFT 的 `disable_adapter()` 或加载时未指定），会用**基座模型**去评测，结果是"训了半天没效果"。

A77 ★★ 怎么判断一个 LoRA 训好了？

A：四层检查，从便宜到贵：

1. **参数校验**：可训练参数占总参数的比例合理（7B + r=16 全线性层 ≈ 0.5%），且非零。
2. **loss 曲线**：训练 loss 应平稳下降、无尖峰；**但 loss 下降不等于能力提升**（见 A57）。
3. **目标任务指标**：在**留出的验证集**上看任务指标（准确率/ROUGE/CIDEr），而不是看 loss。
4. **通用能力回归**：跑一个小的通用评测（如常识问答、指令跟随），确认没有灾难性遗忘——**这是 LoRA 相对全参微调的最大优势，必须验证它真的保住了**。

加分做法：扫 `rank ∈ {8,16,32}` 与 `target ∈ {q/v, 全线性}`，看指标是否饱和——**这能证明你的超参不是拍脑袋的**。

Part B 机器学习算法原理

> 这一部分**不写代码**（代码在手撕两册里），只讲**定义、推导、性质、对比、失效场景**——面试官问"为什么"的时候用。

B1 Softmax 家族（B1–B8）

B1 ★★★ 写出 softmax，它的雅可比矩阵是什么？

A：

```
softmax(z)_i = exp(z_i) / Σ_j exp(z_j)
```

雅可比（这个能推出来很加分）：

```
∂p_i/∂z_j = p_i · (δ_ij − p_j)     即   J = diag(p) − p · pᵀ
```

含义：**对角线是 p_i(1−p_i)（自身对自身的敏感度），非对角线是 −p_i p_j（此消彼长）**，因为输出必须和为 1，抬高一个必然压低其他。

两个关键性质：1. **平移不变**：`softmax(z + c) = softmax(z)`（分子分母同乘 e^c）。 2. **和为 1 且恒正**：输出可解释为概率单纯形上的一点。

B2 ★★★ 为什么要减 max？给出严格的数值论证。

A：`exp(x)` 在 float32 下 x > 88.7 就上溢成 `inf`，`inf/inf = nan`。减 max 后：

```
softmax(z)_i = exp(z_i − m) / Σ_j exp(z_j − m)，其中 m = max_j z_j
```

- **数学等价**：分子分母同乘 `e^{−m}`，结果不变（这是"等价"，不是近似）。
- **数值安全**：减完后最大项是 `exp(0)=1`，其余 ≤ 1，不会上溢；小项下溢成 0 也无害（softmax 里 0 就是"几乎不可能"）。
- **但下溢在 log 域有害**：如果先算 softmax 再取 log，下溢成 0 的地方会变 `−inf`。所以**交叉熵要直接在对数域算**（log_softmax / logsumexp），这是工程上的标准做法。

追问：全 −inf 的一行会怎样？→ 分母 0 → NaN。所以 mask 用 `−1e9` 而不是 `−inf`，或对全屏蔽行单独补 0。

B3 ★★★ 推导 softmax + 交叉熵的梯度为什么是 `p − y`。

A：设 `L = −log p_y = −z_y + log Σ_j exp(z_j)`。

对 `z_i` 求导，分两种情况：

```
∂L/∂z_i = −δ(i=y) + exp(z_i)/Σ_j exp(z_j) = p_i − δ(i=y)
```

即 **`∂L/∂z_i = p_i − y_i`**（y 是 one-hot）。

这个结果的意义：1. 梯度**不含 softmax 的雅可比**——两者精确抵消了（这也是框架把 softmax+CE 合成一个算子（如 `F.cross_entropy`）的原因：更快、更稳）。2. 误差越大梯度越大（预测 0.1、真值 1 → 梯度 −0.9），**没有饱和问题**；3. 对比 MSE + softmax：后者梯度多一个 `p(1−p)` 因子，在"预测 0.99 但真值是 0"这种最该纠错的情况下梯度趋于 0。

B4 ★★ softmax 的温度参数有什么作用?

A：`softmax(z/T)`：

- `T = 1` 标准；
- `T > 1` → 分布更平（熵更大），用于**知识蒸馏**（软标签含"类间相似性"信息）、**采样多样性**、**LLM 生成**；
- `T < 1` → 分布更尖（接近 argmax），用于确定性输出；
- `T → 0` → 退化成 one-hot（hard max）；`T → ∞` → 均匀分布。

为什么蒸馏要用 T：教师模型的原始 logits 里，"猫"和"狗"的相似性会被 softmax 压成接近 0 和 1，T 放大后这部分"暗知识"才显现出来，学生才能学到。

B5 ★★ softmax 存在"过参数化"，怎么理解?

A：`softmax(z + c·1) = softmax(z)` ——**所有 logit 同加一个常数，输出不变**。所以：- C 类分类实际只有 **C−1 个自由度**，参数矩阵存在一整个方向的冗余（解不唯一）；- 导致 Hessian 奇异、优化不唯一、某些正则化行为异常。

三种处理：① 固定某一类的 logit 为 0（等价于用 C−1 组参数，这在数学上恰好等价于 sigmoid 二分类的推广）；② 加 L2 正则（把解拉向唯一）；③ 干脆不管（深度学习里因为正则/优化噪声，实践中不影响）。

B6 ★★ softmax 和 sigmoid 是什么关系?

A：**二分类时两者等价**（差一个常数偏移）：

```
softmax([z, 0])_1 = e^z/(e^z+1) = σ(z)
```

所以"softmax + CE"在 C=2 时与"sigmoid + BCE"给出相同的决策边界和梯度形式。

关键区别在多标签：如果一个样本可以同时属于多个类（不互斥），必须用**逐元素 sigmoid + BCE**，因为 softmax 强制各类概率和为 1（互斥假设）。

B7 ★★ log_softmax 与 logsumexp 技巧是什么?

A：

```
log_softmax(z)_i = z_i − logsumexp(z),   logsumexp(z) = m + log Σ_j exp(z_j − m)
```

- 直接算 `log(softmax(z))` 会先下溢再取 log → `−inf`；log_softmax 全程在对数域，**数值稳定**。
- `logsumexp` 是核心恒等式：**先减最大值，算完再加回来**。它出现在交叉熵、CRF、变分推断、混合模型里。
- 工程上 `torch.logsumexp` 已内置该实现；手写时**务必自己减 max**。

B8 ★ softmax 在什么情况下梯度会消失? 怎么缓解?

A：当某个 logit 远大于其他（`z_max − z_second ≫ 0`）时，softmax 输出接近 one-hot，`p_i(1−p_i) → 0`，**梯度饱和**。这在训练早期 logit 尺度过大时容易出现（初始化不好、学习率过大、输入未归一化）。

缓解：① 输出层用小初始化（或用温度缩放）；② 加 label smoothing（防止一路把概率推到 1）；③ 用 focal loss（主动放大难例权重）；④ 检查输入特征尺度与归一化。**这也是"训练初期 loss 不动"的常见原因之一。**

B2 信息熵与信息论（B9–B16）

B9 ★★★ 信息量和熵的定义与直觉?

A:

- **自信息**：`I(x) = −log p(x)`。概率越小，事件发生带来的"信息量"越大（小概率事件更有信息）。
- **熵**（信息熵）：`H(X) = −Σ_x p(x)·log p(x) = E[−log p(X)]`，即**平均信息量 / 不确定度**。
- 单位：log 以 2 为底是 bit，以 e 为底是 nat（深度学习多用 nat）。
- **极值**：均匀分布熵最大（最不确定）；确定性分布熵为 0。

一句话直觉：熵 = "描述这个随机变量平均需要多少比特" = "我猜它有多难"。

面试常配的追问：为什么熵是不确定度的合理度量？→ 三条公理（连续性、均匀分布最大、独立可加）唯一确定出 `−Σp log p` 的形式（Shannon 唯一性定理）。

B10 ★★★ 联合熵、条件熵、链式法则?

A:

```
联合熵    H(X,Y) = −ΣΣ p(x,y) log p(x,y)
条件熵    H(Y|X) = H(X,Y) − H(X)          （给定 X 后 Y 的剩余不确定度）
链式法则  H(X,Y) = H(X) + H(Y|X)
```

重要性质：- `H(Y|X) ⩽ H(Y)`：知道 X **不会增加** Y 的不确定度（等号当且仅当独立）。- `H(X,Y) ⩽ H(X) + H(Y)`：等号当且仅当独立。

在树模型里怎么用：ID3 的信息增益本质就是 `H(D) − H(D|A)`，即"用特征 A 划分后，标签的不确定度减少了多少"。

B11 ★★★ 互信息是什么？有什么意义?

A:

```
I(X;Y) = H(X) − H(X|Y) = H(Y) − H(Y|X) = H(X) + H(Y) − H(X,Y)
```

含义：知道 Y 能减少多少关于 X 的不确定性（对称的"相关性"度量）。

三个用途（面试可举）：1. **决策树的信息增益** = `I(标签; 特征)`；2. **特征选择**：互信息高 = 该特征与标签相关性强；3. **多模态对齐的另一种视角**：CLIP 的 InfoNCE 本质在最大化**图像与文本的互信息下界**（这是理解对比学习的信息论视角）。

B12 ★★★ KL 散度的定义与三个性质?

A:

```
D_KL(P||Q) = Σ_x p(x)·log( p(x)/q(x) )
```

三个性质：1. **非负**：`D_KL ⩾ 0`，等号当且仅当 P=Q（由 Jensen 不等式或 Gibbs 不等式）。2. **非对称**：`D_KL(P||Q) ≠ D_KL(Q||P)`，所以**不是距离**（不满足三角不等式）。3. **不度量"距离"而度量"代价"**：`D_KL(P||Q)` 是"用 Q 去编码 P 的额外代价"。

非对称的实际后果（这个例子很值分）：设 P 是双峰分布，Q 是单峰高斯：- **前向 `KL(P||Q)`**（最小化 Q 覆盖 P 的所有模式）→ Q 会摊平成宽分布，**mass-covering**；- **反向 `KL(Q||P)`**（Q 避开 P 概率为 0 的地方）→ Q 会塌到某个峰，**mode-seeking**。这就是变分推断（前向）与 GAN/反向 KL 行为差异的根源。

B13 ★★★ 交叉熵和 KL 散度的关系？为什么分类要用交叉熵?

A:

```
H(P,Q) = −Σ p log q = H(P) + D_KL(P||Q)
```

即 **交叉熵 = 真实分布的熵 + KL 散度**。因为 `H(P)` 与模型参数无关（数据固定），所以：

最小化交叉熵 ≡ 最小化 KL 散度，即让预测分布尽量逼近真实分布。

用交叉熵而不用 KL 的工程原因：**不需要知道 H(P)**（P 是 one-hot 时 H(P)=0，直接就是 −log q_y），计算更简单。

一句话：交叉熵是"用模型分布去编码真实标签的代价"，最小化它就是最大似然估计。

B14 ★★★ 信息增益怎么算？它偏向什么？

A：

```
Gain(D, A) = H(D) − Σ_v (|D_v|/|D|)·H(D_v) = H(D) − H(D|A) = I(D; A)
```

即"用特征 A 划分后标签熵的下降量"，也就是互信息。

偏向多值特征：特征取值越多，划分得越细，`H(D|A)` 越容易小（极端情况：用 ID 号划分，每个样本一组，增益最大但毫无泛化）——**这是 ID3 的根本缺陷**，被 C4.5 的增益率修正。

B15 ★★ 信息增益率与固有值是什么？为什么能修正偏向？

A：

```
增益率 = Gain(D, A) / IV(A),   IV(A) = −Σ_v (|D_v|/|D|)·log(|D_v|/|D|)
```

`IV(A)` 是特征 A 自身的**固有值/分裂信息**（把样本按 A 分成 v 组的"熵"），取值越多 IV 越大，于是增益率被惩罚。

但增益率也有反向偏向（偏向取值少的特征），所以 C4.5 用的是**启发式**：先找出增益高于平均的特征，再在其中选增益率最高的。

B16 ★★ 基尼指数和熵是什么关系？为什么 CART 用基尼？

A：

```
基尼    Gini = 1 − Σ_c p_c²   =  Σ_{c≠c'} p_c p_{c'}     （随机取两个样本、类别不同的概率）
熵      H = −Σ_c p_c log p_c
```

- **形状高度相似**：都在均匀分布处取最大，都在纯节点处取 0；二分类下 `Gini = 2p(1−p)`，与熵的曲线几乎重合。
- **基尼是熵的近似**（在均匀分布附近可用低阶展开近似），所以划分结果基本一致。
- **用基尼的原因**：只有乘法加法、**没有 log**，计算快（大数据下差异明显）；且梯度更"平缓"，对噪声略鲁棒。

追问：两者的最优划分会不同吗？→ 可能不同，但实践差异很小；论文级对比显示 mIoU/准确率差异通常在 1% 内。

补充：还有 **最大熵原理**（在已知约束下选熵最大的分布，最"不假设"）和 **困惑度**（`perplexity = exp(H)`，语言模型常用），面试顺口提到很加分。

B3 分类树（B17–B28）

B17 ★★★ ID3 / C4.5 / CART 三种算法怎么对比？

A：

	ID3	C4.5	CART
划分准则	信息增益	**信息增益率**	**基尼指数**（分类）/ MSE（回归）
分裂方式	**多叉**（每个取值一支）	多叉	**二叉**
连续特征	不支持	支持（排序+二分）	支持
缺失值	不支持	支持（按权重分配到各分支）	支持（代理分裂 surrogate split）
剪枝	无	悲观剪枝（PEP）	**代价复杂度剪枝（CCP）**
输出	分类	分类	**分类 + 回归**

一句话："ID3 是原型，C4.5 补工程（连续/缺失/剪枝），CART 补统一性（二叉 + 回归）——sklearn 和所有 GBDT 家族用的都是 CART。"

B18 ★★★ 为什么 CART 用二叉树?

A：四个理由：

1. **避免碎片化**：多叉会让样本被迅速切碎（取值多的特征尤其），每个分支样本太少 → 过拟合且统计不可靠。
2. **表达力更强**：二叉允许**同一特征被多次使用**（"年龄<30 且 年龄>20"），而多叉一个特征只能用一次。
3. **计算高效**：每次只需找最优二分点，比多路划分的组合搜索简单得多。
4. **统一处理类别型特征**：把"取值集合"按 one-vs-rest 二分（对类别特征做分组），天然支持"把 {红,蓝} 归一边"。

B19 ★★★ 连续特征怎么处理?为什么只需枚举相邻中点?

A：做法：该特征的所有取值**排序去重** → 只在**相邻不同取值的中点**试切分。

为什么只需相邻中点：对同一特征，若两个阈值落在同一对相邻取值之间，**划分出的样本集合完全相同**（因为区间内没有样本），所以只有 n−1 个"有效"候选阈值（n = 去重后取值数），彼此等价。

复杂度：排序 O(n log n) + 一次扫描 O(n)（用前缀计数增量更新左右子集的不纯度），而不是"每个阈值重扫一遍"的 $O(n^2)$。

追问：为什么要去重？→ 相同取值的样本任何阈值都分不开，切开是无意义分裂。

B20 ★★★ 剪枝的三种方式?代价复杂度剪枝怎么工作?

A：

方式	做法	优缺点
预剪枝	提前停：深度上限、叶最小样本、增益阈值、纯度阈值	快；但**贪心视野短**，可能过早停止（当前增益小、后续收益大）
后剪枝	先长满，再自底向上判断"是否合并叶子"	更准（看到全树后再决策），但训练成本高
CCP（代价复杂度）	定义 $R_\alpha(T) = R(T) + \alpha \cdot \lvert T \rvert$（R 是训练误差，	T

CCP 的机制：α=0 时最优是全树；α 增大时叶子的"惩罚"变重，逐步剪枝 → 得到一串嵌套子树 → 用验证集选最好的 α（sklearn 的 `ccp_alpha`）。

面试话术："预剪枝省时但短视，后剪枝准但贵；工业上常见组合：先用预剪枝控深度（防止爆内存），再用后剪枝微调。"

B21 ★★ 缺失值怎么处理?

A：两种主流：

- **C4.5**：把缺失样本**按权重分到所有分支**（权重 = 各分支样本比例），增益计算时乘上"有效样本比例"的折扣因子。
- **CART（sklearn 不支持，但理论上有）**：**代理分裂（surrogate split）**——为每个主分裂再学若干"代理特征"，推理时若主特征缺失，用与它最相似的代理特征代替。

工程实践：多数框架直接要求先填充缺失值（均值/中位数/模型预测），再建树——因为树分裂的代理机制实现复杂且收益不稳定。

B22 ★★ 回归树怎么建?叶子输出什么?

A：和分类树完全同构，只换两处：

1. **划分准则**：从基尼换成 **MSE 下降**（$MSE = (1/n)\Sigma(y-\bar{y})^2$）。
2. **叶子输出**：从"多数类"换成 **均值**（最小化平方误差的最优常数就是均值；若损失是 MAE，则取**中位数**）。

注意：回归树的预测是**分段常数**（一个叶子一个值），所以它**不能外推**（超出训练范围的值只能给边界值），且对目标做单调变换会改变结果（因为 MSE 不等变）。

B23 ★★★ 特征重要性怎么算？有什么偏差？

A：两种：

1. **不纯度重要性（Gini/MSE importance，sklearn 默认）**：把"该特征每次分裂带来的加权不纯度下降 × 样本数"累加后归一化。
2. **置换重要性（permutation importance）**：打乱该特征的值（破坏其与标签的关系），看验证指标下降多少。**更可靠、更贵**。

三种已知偏差：- **偏高基数/多值特征**（分裂机会多，累积下降自然大）；- **对相关特征会"分摊"**（两个高度相关的特征各拿一半，看起来都不重要）；- **在不纯度重要性里，训练集上计算会偏向能拟合噪声的特征**。

面试加分："所以我一般用置换重要性做最终判断，不纯度重要性只用来看相对排序。"

B24 ★★ 决策树为什么不稳定？为什么要集成？

A：**不稳定**的原因：树的分裂是**贪心的离散选择**——数据微扰（换一批样本、加几个噪声点）可能让最优切分点改变，后面的整棵子树结构随之变化，预测方差大。

偏差-方差视角：单棵树**偏差低（可以拟合很复杂的边界）、方差高**。所以：- **Bagging / 随机森林**：并行多棵 + 平均 → **降方差**（不显著降偏差）；- **Boosting / GBDT**：串行拟合残差 → **降偏差**（同时要控制方差，所以用小树 + 学习率）。

B25 ★★★ RF / GBDT / XGBoost / LightGBM / CatBoost 怎么串起来？

A：

	集成方式	优化目标	关键创新
随机森林	Bagging（样本 + 特征双采样，并行，投票/平均）	降方差	双重随机性让树之间去相关
GBDT	Boosting（串行，每棵树拟合**负梯度/残差**）	降偏差	用树做梯度下降
XGBoost	GBDT + 工程	二阶泰勒 + 显式正则	二阶信息、正则化（$\gamma T + \lambda w^2/2$）、稀疏感知、特征级并行、近似分位点
LightGBM	GBDT + 效率	同 XGB	**leaf-wise 生长**（同一叶数下更低 loss，但易过拟合，需控 `num_leaves`）、直方图分桶、**GOSS**（保留梯度大的样本）、**EFB**（互斥特征捆绑）
CatBoost	GBDT + 类别特征	同 XGB	**对称树（oblivious）**、**ordered boosting**（解决目标泄漏）、类别特征的有序目标编码

一句话链："RF 是并行降方差，GBDT 是串行降偏差；XGBoost 给 GBDT 加了二阶和正则，LightGBM 把速度做到极致，CatBoost 专治类别特征。"

B26 ★★ XGBoost 相对传统 GBDT 的关键改进？

A：四件事：

1. **二阶泰勒展开**：目标函数用一阶梯度 g + 二阶 h 近似，**最优叶权重有闭式解** $w^* = -\Sigma g/(\Sigma h+\lambda)$，分裂增益也有解析式。这比只用一阶梯度收敛更快、更准。
2. **显式正则**：$\Omega = \gamma T + (\lambda/2)\Sigma w^2$，直接在目标函数里惩罚树复杂度（叶数 T 和叶权重 w），而不是靠早停/剪枝。
3. **稀疏感知**：为缺失值学一个默认方向（分裂时把缺失样本统一分到增益大的一侧）。
4. **工程**：特征预排序（block 结构）、特征级并行、近似分位点（`approx_hist`）、缓存优化。

B27 ★★ LightGBM 的 leaf-wise、GOSS、EFB 分别解决什么？

A：

- **leaf-wise（按叶生长）**：每次选"增益最大的叶子"继续分裂，而不是按层。同样的叶数下 loss 更低（效率高），但**容易长出深的不平衡树 → 过拟合**，所以要用 `num_leaves` 和 `max_depth` 双重约束。

- **GOSS（Gradient-based One-Side Sampling）**：梯度大的样本对增益贡献大，**保留全部大梯度样本 + 随机采样一部分小梯度样本**（并放大权重），用少量样本近似全量增益。
- **EFB（Exclusive Feature Bundling）**：稀疏场景下很多特征互斥（不会同时非零），把它们**捆绑成一个特征**，降低特征维度 → 减少直方图构建开销。

B28 ★ 什么时候选树模型，什么时候选线性/神经网络?

A:

场景	选树	选线性/NN
表格数据、中小规模、需要可解释	✅树（尤其 GBDT，仍是表格任务的王者）	—
高维稀疏（文本 TF-IDF）	—	✅线性（快、可解释）
特征有强交互、非线性	✅树	✅NN（需更多数据）
需要外推（预测超出训练范围）	❌树不能外推	✅线性/NN
需要平滑输出	❌分段常数	✅
图像/语音/文本原始信号	—	✅深度模型

一句话："树赢在'不需要特征工程 + 自动做交互 + 可解释'，输在'不能外推、不平滑、不稳定'。"

B4 K-Means（B29–B36）

B29 ★★★ KMeans 的算法与目标函数?

A:

目标函数（inertia / 簇内平方和）：

```
J(C, μ) = Σ_i min_j ||x_i − μ_j||²
```

算法（交替优化 / 坐标下降）：

```
1. 初始化 k 个中心
2. E 步（固定中心）：把每个点分给最近的中心
3. M 步（固定分配）：把每个中心更新为其簇内所有点的均值
4. 重复 2-3 直到中心移动量 < tol 或达到迭代上限
```

为什么 M 步是均值：固定分配后，`J = Σ_j Σ_{i∈C_j} ||x_i − μ_j||²` 对 μ_j 求导置零，得 `μ_j = (1/|C_j|)Σ x_i` ——即**均值是最小化平方误差的最优常数**。

B30 ★★★ 为什么 KMeans 一定收敛？为什么只是局部最优?

A:

收敛：E 步（在固定中心下，把点分给最近中心）不会增大 J；M 步（固定分配下取均值）也不会增大 J。J 单调不增且有下界 0；而划分方案是**有限个**（n 个点分到 k 个簇，最多 k^n 种，实际远少），所以不会无限循环 → **有限步内收敛到一个驻点**。

局部最优：因为 J 非凸，交替优化只保证到达**坐标下降的局部极小**。初始化不同 → 收敛到不同的局部解（我实测一组数据：20 个随机种子里有 7 个没能正确分出三个簇，最差的只对上 125/150）。

工程对策：`n_init` 多次随机初始化取 inertia 最小；或 k-means++ 初始化。

B31 ★★★ k-means++ 的原理？为什么有效?

A：**D^2 采样**：

```
1. 第一个中心均匀随机选一个点
2. 之后每个新中心按概率 ∝ D(x)² 采样（D(x) = 到最近已有中心的距离）
3. 重复到选出 k 个
```

即"**离已有中心越远的点，越容易被选为新中心**"。

为什么有效：随机初始化最坏情况可能把两个中心都放在同一个真实簇里（剩下一个簇分不到中心），D^2 采样让中心天然分散，规避了这种退化。

理论保证：k-means++ 的期望代价 ≤ **O(log k)** 倍最优解（对比随机初始化的任意差）。代价只是初始化时多算几次距离。

B32 ★★★ k 怎么选？（四种方法）

A：

1. **肘部法（elbow）**：画 inertia-k 曲线，找拐点（边际收益骤降处）。缺点是"肘"常常不明显。
2. **轮廓系数（silhouette）**：`s = (b − a)/max(a, b)`，a = 到本簇其他点的平均距离，b = 到最近其他簇的平均距离。范围 [−1,1]，越大越好，取 s 平均最大的 k。
3. **Gap statistic**：比较 log(inertia) 与"零假设下（均匀分布）的期望值"的差，差距最大处为最优 k。
4. **稳定性 / 下游指标**：多次子采样聚类的一致性；或直接用下游任务指标（最实际）。

面试加分："我更看重**稳定性**——k 稍微变化时簇结构是否稳定；如果 k=4 和 k=5 的划分看起来完全不同，说明数据本身没有清晰的簇结构。"

B33 ★★ KMeans 的复杂度？怎么加速？

A：**O(n · k · d · t)**（n 样本、k 簇、d 维、t 迭代轮数）。空间 O(n · d + k · d)。

加速手段： 1. **MiniBatch KMeans**：每步只用一个小 batch 更新中心（`O(b · k · d · t)`），大规模数据首选。 2. **Elkan 算法**：用三角不等式剪枝，跳过不可能更近的中心计算。 3. **kd-tree / ball-tree**：低维（d < 20）时加速最近邻搜索。 4. **降维**（PCA/随机投影）后再聚类。 5. **初始化用 k-means++ + n_init 小**（减少迭代次数）。

B34 ★★ KMeans 的假设与失效场景？

A：四个隐含假设：

1. **簇是凸的、球形的**（欧氏距离隐含等向性）→ 遇到月牙形/环形数据直接失效（该用 DBSCAN/谱聚类）。
2. **各簇方差相近、大小相近** → 一个大簇会被劈成两半去迁就小簇。
3. **欧氏距离有意义** → 高维下距离集中（所有点距离都差不多），聚类失效。
4. **对异常值敏感** → 均值被拉走（可用 k-medoids / 截断损失）。

另外：它只给**硬分配**，不能表达"某个点 60% 属于簇 A"（要软分配用 GMM）。

B35 ★★ KMeans / GMM / DBSCAN / 层次 / 谱聚类怎么选？

A：

方法	核心思想	优点	缺点
KMeans	中心 + 硬分配	快、简单、可扩展	球形假设、要指定 k、异常敏感
GMM（EM）	高斯混合 + 软分配	概率输出、允许不同协方差（椭圆簇）	要指定 k、对初始化敏感、高斯假设
DBSCAN	密度可达	不用指定 k、能找任意形状、能识别噪声	对 eps/minPts 敏感、密度差异大时失效、高维差
层次聚类	自底向上合并 / 自顶向下分裂	不用指定 k、给树状结构、可解释	$O(n^2)$/$O(n^3)$、不可回溯（贪心合并）
谱聚类	图拉普拉斯特征分解	能处理非凸簇	要指定 k、大图贵（需稀疏近似）

KMeans 与 GMM 的关系（高频追问）：KMeans 是 GMM 的**特例**——所有簇共享各向同性协方差 $\sigma^2 I$ 且 $\sigma \to 0$ 时的硬分配极限。EM 的 E 步是软分配（后验概率），M 步更新均值/协方差/权重。

B36 ★ 空簇和初始化有哪些实际陷阱?

A：三个：

1. **空簇**：某轮没有点分给它 → 均值无定义（除零）。处理：保留旧中心、或把它重置到"离其他中心最远的点"、或直接删掉该簇（k 减一）。
2. **初始中心重复**：随机抽到同一个点（k 接近 n 时概率不低）→ 退化。用 `replace=False` 抽样或 k-means++。
3. **用整数存中心**：均值可能是小数，用 int 会累积误差甚至震荡不收敛（这是实现 bug，但很常见）。

收敛判据的选择：用"中心移动量 < tol"而不是"标签不再变化"——后者在某些数据上会出现**标签循环震荡**（目标函数不变但分配在两组等价解之间跳）。

B5 自注意力（B37–B45）

B37 ★★★ 写出 self-attention 的完整定义，并说明每一步的形状。

A：

```
Q = X · W_Q,  K = X · W_K,  V = X · W_V          X: [L, d],  W: [d, d]
A = softmax( Q · Kᵀ / √d_k + mask , dim=-1)  Q · Kᵀ: [L, L]
O = A · V                                   [L,L]×[L,d] → [L, d]
```

三步的语义： 1. **打分**：`Q · Kᵀ` 得到任意两 token 的相似度（**全局**、与距离无关）； 2. **归一化**：逐行 softmax → 每个 query 对所有 key 的注意力分布（和为 1）； 3. **聚合**：用注意力权重对 V 加权求和 → 每个位置得到"按相关性混合"的新表示。

三个矩阵的不同角色：Q 是"我要找什么"，K 是"我能被什么匹配"，V 是"匹配上之后提供什么内容"。

B38 ★★★ 为什么除以 √d_k? 给出方差论证。

A：设 `q, k` 各维独立同分布、均值 0、方差 1，则点积：

```
q · k = Σ_{i=1}^{d_k} q_i · k_i
E[q · k] = Σ E[q_i]E[k_i] = 0
Var[q · k] = Σ Var(q_i k_i) = Σ E[q_i²] · E[k_i²] = d_k        (独立性 + 零均值)
```

所以点积的**标准差是 √d_k**——d_k=64 时量级约 8，d_k=256 时约 16。softmax 的输入尺度一大，就会进入**饱和区**（最大项≈1、其余≈0），此时雅可比 `diag(p)−ppᵀ ≈ 0`，**梯度消失**。

除以 √d_k 后方差回到 1，softmax 落在梯度良好的区间。**这也是 d_k 越大越需要缩放的原因。**

B39 ★★★ self-attention 的复杂度? 优化方向有哪些?

A：

```
时间: O(L² · d)      (打分 L²d + 加权 L²d)
显存: O(L² + L · d)  (要存注意力矩阵 L²)
```

L 是序列长度，所以**长上下文是平方级代价**。五个优化方向：

1. **IO 优化**：**FlashAttention**——分块（tiling）+ 在线 softmax，不把 L×L 矩阵落到显存，**省显存和 HBM 读写，不减少 FLOPs**。
2. **稀疏/滑窗**：Longformer（滑窗 + 全局 token）、BigBird（随机 + 窗口 + 全局）。
3. **低秩/核近似**：Linformer（对 K/V 降维）、Performer（核函数近似 softmax）。
4. **共享 KV**：MQA/GQA（主要省推理 KV cache）。
5. **换范式**：线性注意力、状态空间模型（Mamba，线性复杂度）。

B40 ★★★ 多头的作用是什么? 参数量怎么算?

A：把 `d` 维按**列**切成 h 份，每份 `d_k = d/h`，各自做 attention，再把 h 个 `[L, d_k]` 输出**按列拼回** `[L, d]`，最后过输出投影 `W_O`。

作用：不同头在**不同的子空间**学不同的依赖模式（相邻、句法、指代、跨模态对应），类似 CNN 的多通道分工。**计算量几乎不变**（$h \cdot L^2 \cdot (d/h) = L^2 \cdot d$），只是注意力的"视角"变多。

参数量：$4d^2$（Q/K/V/O 各 d×d）+ bias $4d$，**与 h 无关**（这是常被考的常识）。

B41 ★★★ 为什么需要位置编码？四种方案的取舍？

A：**self-attention 是置换等变的**（打乱输入只是打乱输出行），本身不含顺序信息，所以必须显式注入位置。

方案	做法	优点	缺点
正弦绝对编码	不同频率的 sin/cos	无需学习、可外推到更长	表达力弱、外推质量差
可学习绝对编码	每个位置一个向量	简单、效果好	**不能外推**（超过训练长度没有参数）
RoPE（旋转）	把 q/k 的相邻两维按位置旋转	编码**相对**位置、可外推、无额外参数	外推需插值/NTK 调整
ALiBi	注意力分数按距离加线性负 bias	外推好、实现极简、无参数	表达力略弱于 RoPE

RoPE 为什么能外推：因为 $q_m \cdot k_n$ 只依赖相对距离 $(m-n)$，所以理论上长度无关；但直接外推到远超训练的尺度会失真，所以实践要配位置内插（PI）、NTK-aware 缩放、YaRN。

多模态的扩展：图像/视频需要 2D/3D 位置（高、宽、时间），如 Qwen-VL 的 **M-RoPE**（把旋转维度分给 t/h/w）。

B42 ★★ attention 里的 mask 有哪几种？实现要注意什么？

A：三种：

1. **causal mask**（因果/自回归）：位置 i 只能看到 ⩽ i 的 key → 上三角置 $-\infty$。
2. **padding mask**：一个 batch 里不同的有效长度，key 侧的 pad 位置屏蔽。
3. **结构/跨模态 mask**：如视锥裁切（BEV query 看不到的相机）、跨样本隔离。

实现注意：- 用加性 mask（$scores + (-1e9 \text{ 或 } -inf)$）或 $masked_fill$，**统一语义**（True 是屏蔽还是保留，PyTorch 的 SDPA 里 bool mask 是 $True = 允许$，与 $masked_fill$ 相反）。- **一行全被屏蔽 → 分母为 0 → NaN**：要么保证每行至少一个有效位，要么对全屏蔽行单独处理。

B43 ★★ "置换等变"具体是什么意思？有什么后果？

A：设 P 是置换矩阵，则 $Attention(PX) = P \cdot Attention(X)$ ——**输入换顺序，输出只是跟着换顺序，内容不变**。

两个直接后果：1. **必须加位置编码**（否则模型分不清"猫追狗"和"狗追猫"）；2. **注意力的权重矩阵不含"距离"概念**（不像卷积的局部性）——所以它对"相邻"这件事没有先验，全靠学。

顺带一个面试题：为什么 Transformer 能表达卷积？→ 可以用特定的注意力模式（固定的相对位置偏置）模拟卷积核的行为，即"Transformer 的表达力包含卷积"。

B44 ★★ MQA / GQA / FlashAttention / 线性注意力 / Mamba 各解决什么？

A：

方法	解决什么	代价
MQA（Multi-Query）	所有头共享 1 组 K/V → KV cache ÷h	表达力略降
GQA（Grouped-Query）	h 个头分 g 组共享 K/V → ÷(h/g)	折中，主流选择
FlashAttention	分块 + 在线 softmax，不落 L^2 矩阵 → **省显存与 HBM 读写**	算法等价，不改 FLOPs
线性注意力（Performer/Linformer）	核近似/低秩 → **O(L) 复杂度**	近似误差、长程依赖略弱
Mamba / SSM	选择性状态空间 → 线性复杂度、推理 O(1) 每步	表达力待验证、生态不成熟

一句话："MQA/GQA 省的是**推理时的 KV cache**；FlashAttention 省的是**显存和带宽**；线性注意力/Mamba 改的是**复杂度阶**——三者解决的是不同层面的问题。"

B45 ★ self-attention 与 CNN / RNN 的本质区别?

A:

	CNN	RNN	Self-Attention
感受野	局部（靠堆叠扩大）	理论上全局（实际衰减）	**每层全局**
并行性	高	**低**（时序依赖）	高
归纳偏置	强（局部性、平移等变）	中（序列性）	**弱**（几乎无先验）
复杂度	$O(L \cdot k^2)$（与长度线性）	O(L)（串行）	**$O(L^2)$**
长依赖	需深层	梯度消失/爆炸	直接建模（但可能"注意力分散"）

取舍结论："**归纳偏置越强，小数据越好、大数据越受限**。CNN 在数据少时更强，Transformer 在数据够时上限更高——这也是 ViT 需要大数据/强增广/蒸馏才超过 CNN 的原因。"

B6 用代码验证原理（Python 语法不熟也能跑）

> 这一节的目的不是"实现算法"，而是**用几行代码把抽象结论变成看得见的数字**。 例如"softmax+CE 的梯度是 p−y"——写 5 行数值微分就能验出来，比背公式记得牢。
>
> 完整可运行文件： `ml_principles_verify.py` （47 项断言，全部实测通过），运行 `python3 ml_principles_verify.py` 。 下面每个片段都是那个文件里的精简版，**可以直接复制到 Python 里跑**。

B6.0 先补 12 个语法点（这一节只用得到这些）

你只写 C++ 的话，下面这些是"看得懂这段代码"的最小集：

#	语法	说明	等价 C++
1	`np.array([1,2,3])`	列表 → 数组	`vector<int>{1,2,3}`
2	`x.max(axis=-1, keepdims=True)`	沿最后一维取最大； `keepdims` 保住维度	手写循环 + 记录维度
3	`a * b` / `a / c`	**逐元素**运算，自动广播	循环逐元素
4	`y[mask]`	**布尔索引**：取出 mask 为 True 的元素	`if (mask[i]) v.push_back(...)`
5	`~mask`	布尔取反（**不是 `!`**）	`!mask`
6	`a[:, j]`	逗号左边是行、右边是列； `:` 表示全部	手写循环取第 j 列
7	`a[::-1]`	反向切片	`reverse`
8	`np.unique(a)`	去重并排序	`sort` + `erase(unique)`
9	`np.bincount(y)`	统计每个整数出现几次	`int cnt[k]={0}` 累加
10	`f"{x:.4f}"`	f-string 格式化， `.4f` 是 4 位小数	`printf("%.4f", x)`
11	`[... for x in xs]`	列表推导式	`for` + `push_back`
12	`np.allclose(a, b, atol=1e-6)`	浮点近似相等（**自检必用**）	`fabs(a-b)<eps`

一个提醒：Python 的 `=` 是**引用**， `b = a` 不会复制数组，要复制写 `b = a.copy()` 。这在本节里出现过好几次，是最容易踩的坑。

B6.1 验证 softmax 的三条结论

① 雅可比矩阵就是 $\mathrm{diag}(p) - p \cdot p^T$

```python
import numpy as np, math

def softmax(z):
    z = np.asarray(z, dtype=np.float64)        # ← 语法点：list 转数组
    z = z - z.max(keepdims=True)               # 减最大值（数值稳定，见下一条）
    e = np.exp(z)
    return e / e.sum(keepdims=True)

z = np.array([0.5, -0.2, 1.1])
p = softmax(z)

J_analytic = np.diag(p) - np.outer(p, p)       # ← 语法点：np.outer 求外积 → 3×3

eps = 1e-6
J_numeric = np.zeros((3, 3))
for j in range(3):                             # ← 语法点：range(3) = 0,1,2
    z2 = z.copy()                              # ← 语法点：必须 copy，否则改的是同一个数组
    z2[j] += eps
    J_numeric[:, j] = (softmax(z2) - p) / eps  # ← 语法点：[:, j] 取第 j 列

print(np.allclose(J_analytic, J_numeric, atol=1e-5))     # True
print(J_analytic.sum(axis=1))                            # [0, 0, 0]
```

跑出来的两件事：1. 解析雅可比与数值微分**完全一致** → 公式 $\partial p_i/\partial z_j = p_i(\delta_{ij} - p_j)$ 是对的；2. 雅可比**每行和为 0** → 这就是"输出必须和为 1，所以所有 logit 同加常数不变"的数学体现。

② 不减 max 会直接得到 nan

```python
z = np.array([1000.0, 1000.0, 1000.0])
naive = np.exp(z) / np.exp(z).sum()     # RuntimeWarning: overflow → nan
safe  = softmax(z)                       # [1/3, 1/3, 1/3]
```

观察：`np.exp(1000)` 超 float64 上限 → `inf`，`inf/inf = nan`。减 max 后最大项是 `exp(0)=1`，永不溢出。

③ 交叉熵的梯度就是 $p - y$

```python
z = np.array([0.3, 1.2, -0.7]); y = 1

def loss(z):
    return -math.log(softmax(z)[y])        # ← 语法点：函数里可以直接用外部变量 y

p = softmax(z)
grad_analytic = p.copy(); grad_analytic[y] -= 1          # ← p − y

eps = 1e-6
grad_numeric = np.zeros(3)
for i in range(3):
    zp, zm = z.copy(), z.copy()
    zp[i] += eps; zm[i] -= eps
    grad_numeric[i] = (loss(zp) - loss(zm)) / (2 * eps)   # 中心差分

print(np.allclose(grad_analytic, grad_numeric, atol=1e-6))   # True
```

这就是"框架为什么把 softmax 和 CE 合成一个算子"的实证：两者的雅可比精确抵消，$p - y$ 既简单又不饱和。

顺带两个小验证（一行代码的结论）

```python
# 温度：T 越大分布越平
for T in (0.5, 1.0, 3.0):
    print(T, np.round(softmax(z / T), 4))

# 二分类时 softmax([z,0]) == sigmoid(z)
print(softmax([1.7, 0.0])[0], 1 / (1 + math.exp(-1.7)))    # 0.8455 0.8455
```

B6.2 验证信息熵的四条结论

① 链式法则：`H(X,Y) = H(X) + H(Y|X)`

```
def entropy(p):
    p = np.asarray(p, dtype=np.float64)
    p = p[p > 0]                              # ← 语法点：布尔索引筛掉 0（0 · log0 记为 0）
    return -(p * np.log(p)).sum()

Pxy = np.array([[0.20, 0.05, 0.00],
                [0.05, 0.30, 0.05],
                [0.00, 0.05, 0.30]])
Px = Pxy.sum(axis=1)                          # 按行求和 = X 的边缘分布
Hxy = entropy(Pxy.ravel())                    # ← 语法点：ravel 把矩阵摊平
Hx = entropy(Px)
Hy_given_x = sum(Px[i] * entropy(Pxy[i] / Px[i]) for i in range(3))
#            ^ ↑ 语法点：生成器表达式放进 sum()，等价于一个 for 循环累加
print(abs(Hxy - (Hx + Hy_given_x)) < 1e-12)   # True
```

为什么值得跑一遍：条件熵的定义式 `H(Y|X) = Σ_x p(x)·H(Y|X=x)` 里"**每一项都要先归一化**（`Pxy[i] / Px[i]`）"这个细节，光看公式容易漏；跑一次就记住了。

② KL 散度非对称

```
P = np.array([0.5, 0.5])
Q = np.array([0.9, 0.1])

def kl(a, b):
    m = a > 0
    return float((a[m] * np.log(a[m] / b[m])).sum())

print(kl(P, Q), kl(Q, P))      # 0.5108  1.2451  ← 差别很大
```

观察到的两个数：`KL(P||Q)≈0.51`、`KL(Q||P)≈1.25`——这直接说明它**不是距离**（不满足对称性），也解释了变分推断里"前向 KL 会让 Q 摊平覆盖 P 的所有峰，反向 KL 会塌到某个峰"。

③ 交叉熵 = 熵 + KL

```
ce = -float((P * np.log(Q)).sum())
h  = entropy(P)
kl_v = float((P * np.log(P / Q)).sum())
print(abs(ce - (h + kl_v)) < 1e-12)     # True
```

结论：分类时 `H(P)` 是常数，所以**最小化交叉熵 ≡ 最小化 KL 散度**——这就是"为什么用交叉熵而不是直接算 KL"（省掉算 H(P)）。

④ 信息增益偏向多值特征（最重要的实验）

```
y = np.array([1, 1, 1, 0, 0, 0])          # 3 正 3 负
A = np.array([0, 0, 0, 1, 1, 1])          # 好特征：2 个取值，每类纯
ID = np.arange(6)                          # 坏特征：每个样本一个取值

def cond_entropy(feat, y):
    total = 0.0
    for v in np.unique(feat):              # ← 语法点：np.unique 去重
        mask = feat == v
        w = mask.sum() / len(feat)
        total += w * entropy(np.bincount(y[mask], minlength=2) / mask.sum())
    return total

H_parent = entropy([0.5, 0.5])                      # 0.6931
gain_A  = H_parent - cond_entropy(A,  y)            # 0.6931  ← 完全分开
gain_ID = H_parent - cond_entropy(ID, y)            # 0.6931  ← 也"完全分开"!
```

这就是 ID3 缺陷的现场证据：用"样本 ID"这种毫无泛化能力的特征，信息增益同样是最大的。接着验证增益率能区分它们：

```
def iv(feat):
    return entropy(np.array([(feat == v).sum() / len(feat) for v in np.unique(feat)]))

print(gain_A / iv(A), gain_ID / iv(ID))    # 1.0000   0.3869
```

结论：增益率把 ID 号特征从 1.0 压到 0.387——因为它的"分裂信息 IV"更大（6 个取值 vs 2 个取值）。

B6.3 验证分类树的两条结论

① 手写 CART 能拟合到 100%（用来确认实现没错）

```
def gini_counts(y, K):
    p = np.bincount(y, minlength=K) / len(y)      # ← 语法点：bincount 统计每类次数
    return 1.0 - (p ** 2).sum()

def build_tree(X, y, depth=0, max_depth=4, min_leaf=2):
    node = {"label": int(np.bincount(y, minlength=2).argmax())}   # ← 用 dict 当节点
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_leaf:
        return node
    best = (1e-9, None)
    parent = gini_counts(y, 2)
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        for t in (vals[:-1] + vals[1:]) / 2:       # ← 候选阈值 = 相邻中点
            m = X[:, f] <= t
            if m.sum() < min_leaf or (~m).sum() < min_leaf:
                continue
            g = parent - (m.sum() * gini_counts(y[m], 2)
                          + (~m).sum() * gini_counts(y[~m], 2)) / len(y)
            if g > best[0]:
                best = (g, (f, t, m))
    if best[1] is None:
        return node
    f, t, m = best[1]
    node.update({"feat": f, "thr": float(t),
                 "left":  build_tree(X[m],  y[m],  depth + 1, max_depth, min_leaf),
                 "right": build_tree(X[~m], y[~m], depth + 1, max_depth, min_leaf)})
    return node
```

注意三个 Python 语法点：- `{"label": ...}` 用**字典**表示节点——Python 没有指针，字典/对象都是"引用传递"，所以 `node["left"] = build_tree(...)` 天然就能建树（等价于 C++ 的 `node->left = ...`，但不用管内存）。- `~m` 是布尔取反（`!` 在 numpy 里不适用）。- `vals[:-1] + vals[1:]` 这个切片加法一次就得到所有相邻对，C++ 里要显式循环。

实测：两簇数据训练集准确率 **1.000**，根节点阈值切在两簇之间。

② 连续特征的候选阈值只有 n−1 个

```
col = np.array([0.1, 0.5, 0.5, 0.9, 1.7])
vals = np.unique(col)                    # [0.1, 0.5, 0.9, 1.7] —— 5 个点去重成 4 个
thrs = (vals[:-1] + vals[1:]) / 2        # [0.3, 0.7, 1.3] —— 3 个候选
print(len(thrs) == len(vals) - 1)        # True
```

说明：原始 5 个样本、去重后 4 个不同值 → 只有 **3 个有效候选阈值**。两个 0.5 之间切不开，任何落在同一区间的阈值划分结果完全相同。

B6.4 验证 K-Means 的三条结论

① inertia 单调不增（收敛性的证据）

```
def kmeans_simple(X, k, iters=50, seed=0, record=False):
    rng = np.random.default_rng(seed)                    # ← 语法点：新式随机数生成器
    C = X[rng.choice(len(X), k, replace=False)].copy()   # ← 选 k 个不同下标
    hist = []
    for _ in range(iters):                               # ← 语法点：_ 表示"用不到的循环变量"
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)   # [N,k] 距离平方（广播）
        lab = d.argmin(1)                                # 每行最小值的下标
        for j in range(k):
            m = lab == j
            if m.any():                                  # ← 语法点：.any() 有 True 就 True
                C[j] = X[m].mean(0)                      # 布尔索引取子集再求均值
        hist.append(((X - C[lab]) ** 2).sum())           # C[lab] = 按标签取中心
    return lab, C, hist
```

实测输出：首轮 inertia ≈ 890 → 末轮 ≈ 98，**每轮都不增**（`np.diff(hist) <= 0` 全为真）。这就是 B30 说的"E 步和 M 步都不会增大目标函数"的实证。

② 局部最优：20 个种子差 55 倍

```
inertias = [kmeans_simple(X3, 3, seed=s)[2][-1] for s in range(20)]
inertias = sorted(inertias)
print(f"min={inertias[0]:.1f} 中位={inertias[10]:.1f} max={inertias[-1]:.1f}")
```

实测：`min=22.7 / 中位=22.7 / max=1247.2`——最差的种子比最好的**差 55 倍**，因为它把两个真实簇合并、另一个簇劈开了。

这句话可以直接讲给面试官："同一份数据、同一个算法，只换初始化，inertia 能差两个数量级——所以 `n_init` 或 k-means++ 不是优化项，是必需品。"

③ 球形假设失效：同心环上完全分不开

```
t = np.linspace(0, 2 * math.pi, 100, endpoint=False)     # ← 语法点：等距数列
ring1 = np.column_stack([np.cos(t), np.sin(t)]) * 1.0     # ← 按列拼成 (100,2)
ring2 = np.column_stack([np.cos(t), np.sin(t)]) * 3.0
X = np.vstack([ring1, ring2])                             # ← 语法点：vstack 竖着拼
```

实测：kmeans(k=2) 在同心环上的"按环划分纯度"只有 **100/200**（等于完全没分开：每个簇里两个环各占一半）。这就验证了"kmeans 只能找凸的、球形的簇"——这种数据该用 DBSCAN 或谱聚类。

顺带实测了 k 的选择：

k	inertia	轮廓系数均值
1	887.8	—
2	**98.3**	**0.778**
3	81.4	0.548
4	68.3	0.566

读法：inertia 从 k=1 到 k=2 掉了一大截（887→98），之后平缓 → **肘部在 k=2**；轮廓系数也在 k=2 最高。两种方法给出同一答案，这才是可信的 k。

B6.5 验证自注意力的四条结论

① 手算 attention，打印每个中间量

```
def attention_manual(X, Wq, Wk, Wv, mask=None):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # ← @ 就是矩阵乘
    dk = Q.shape[-1]                          # 最后一维长度
    S = Q @ K.T / math.sqrt(dk)               # [L,L] 相似度
    if mask is not None:
        S = np.where(mask, -1e9, S)           # ← 逐元素三目
    A = softmax(S)                            # 逐行归一化
    return A @ V, A
```

取 `X=[[1,0],[0,1],[1,1]]`、权重为单位阵，实测中间量：

```
Q·Kᵀ     = [[1, 0, 1],        scores = Q·Kᵀ/√2 = [[0.707, 0,    0.707],
            [0, 1, 1],                            [0,     0.707,0.707],
            [1, 1, 2]]                            [0.707, 0.707,1.414]]
attn     = [[0.4011, 0.1978, 0.4011],   行和 = [1, 1, 1]
            [0.1978, 0.4011, 0.4011],
            [0.2483, 0.2483, 0.5035]]
```

看懂这张表：第 3 行（token 是 [1,1]）和所有 token 的相似度都高，所以注意力更"平均"；第 1、2 行各自只跟自己像，于是对角权重最大。**这就是"注意力按相关性分配"的直观样子**。

② 为什么除以 √dk：实测标准差

```
for dk in (8, 64, 256):
    q = rng.normal(size=(2000, dk)); k = rng.normal(size=(2000, dk))
    dots = (q * k).sum(1)                    # 逐行点积
    print(dk, dots.std(), (dots / math.sqrt(dk)).std())
```

实测：

dk	未缩放 std	缩放后 std
8	2.76（≈√8=2.83）	0.98
64	7.94（≈√64=8）	0.99
256	16.15（≈√256=16）	1.01

这就是 B38 方差论证的实证：`Var[q·k] = d_k`、`std = √d_k`；除以 √dk 后所有维度的 scores 都回到 std≈1，softmax 不会饱和。**面试时能报出这三个数字，比只背公式强得多。**

③ 置换等变：打乱输入，输出同序打乱

```
out,      _ = attention_manual(X, W, W, W)
perm = np.array([2, 0, 3, 1])                # ← 语法点：用数组当下标做重排
out_perm, _ = attention_manual(X[perm], W, W, W)
print(np.allclose(out_perm, out[perm]))       # True
```

结论：attention 对输入顺序**完全无感**——所以必须加位置编码（B41）。这是"为什么非要 RoPE/正弦编码"的最硬证据。

④ 因果 mask 的实际权重矩阵

```
mask = np.triu(np.ones((4, 4), dtype=bool), k=1)   # ← 语法点：np.triu 取上三角
```

实测输出：

```
attn = [[1.000, 0.000, 0.000, 0.000],
        [0.001, 0.999, 0.000, 0.000],
        [0.109, 0.651, 0.240, 0.000],
        [0.080, 0.517, 0.172, 0.231]]     每行和都 = 1
```

读法：上三角全 0（看不到未来），下三角按相关性分配；每行仍然和为 1（归一化只在可见范围内做）。顺便能看到"越靠后的 token 分给第一个 token 的权重越小"，这是因为 softmax 重新归一化后的自然结果。

B6.6 这一节能给你什么（面试用法）

1. **被问"你怎么确认自己理解对了"**：答"我习惯用数值微分/小数据把公式跑出来验证，比如 softmax 的雅可比我用数值微分对过，交叉熵的梯度确实是 p−y"。
2. **被问"为什么除 √dk"**：直接报实测——"dk 从 8 到 256，点积标准差从 2.76 涨到 16.15，正好是 √dk"。
3. **被问"kmeans 为什么只是局部最优"**：报"同一数据 20 个初始化，inertia 从 22.7 到 1247，差 55 倍"。
4. **被问"信息增益有什么问题"**：现场写三行——用 ID 号特征算出和信息增益一样大的结果，再报增益率把它从 1.0 压到 0.387。

这四句话的共同点：都是"我跑过"而不是"我记得"。

Part C 图像处理与卷积网络

> 这一部分补 CV 基本功：**Part C1 图像处理**（C1–C12）→ **C2 CNN 组件与计算**（C13–C26）→ **C3 架构演进**（C27–C36）→ **C4 训练调优**（C37–C44）→ **C5 指标与损失**（C45–C50）→ **C6 与传统/多模态的接口**（C51–C54）。 特点：**凡是能算的都给出算式和具体数字**，面试时现场算一遍比背结论有说服力。

C1 图像处理基础（C1–C12）

C1 ★★ 一张图像在计算机里怎么表示？常见色彩空间各适合什么？

A：表示：`H×W×C` 的张量，`dtype` 常为 uint8（0–255）或 float32（归一化到 0–1 或 z-score）。

色彩空间	含义	适合什么
RGB	红绿蓝加色	显示、深度学习默认输入（但三通道高度相关）
HSV / HSL	色相、饱和度、明度	**颜色分割、颜色增广**（改色相不影响亮度）
YUV / YCbCr	亮度 + 两路色度	**视频编码/压缩**（人眼对亮度更敏感，色度可下采样 4:2:0）
Lab / CIELAB	接近人眼感知均匀	颜色差计算（ΔE）、跨光照的稳定性
灰度	单通道亮度	边缘/纹理/OCR 预处理

面试考点：为什么视频压缩用 YUV？→ 因为可以**对色度下采样**（4:2:0 直接省 50% 数据）而人眼几乎无感，RGB 做不到这一点（三通道同等重要）。

C2 ★★★ resize 用了什么插值？对下游有什么影响？

A：三种主流插值：

方法	做法	特点
最近邻	取最近的源像素	最快、无新值；**放大有锯齿，缩小会丢信息（欠采样别名）**
双线性	4 邻域加权（按距离）	快、平滑；**缩小会模糊**（相当于低通）
双三次	16 邻域（三次核）	质量最好、边缘更锐；稍慢

工程要点（面试常问）： - **缩小图像要先抗锯齿**（area 插值或先高斯模糊再采样），否则出现摩尔纹/别名—— `cv2.INTER_AREA` 就是干这个的。 - **深度学习里 resize 用双线性居多**（与 backbone 的平滑性匹配），**语义分割最好用双线性/双三次**（最近邻会让 mask 边界错位）。 - **全卷积网络对分辨率敏感**：训练用 512、测试用 1024 会掉点，因为感受野/统计量变了。

C3 ★★★ 常见的图像滤波有哪些？各自适合去什么噪声？

A：

滤波器	原理	适合	代价/缺点
均值滤波	邻域平均	高斯噪声	边缘模糊严重
高斯滤波	按距离加权（σ 控制平滑度）	高斯噪声、平滑	边缘也糊；σ 越大越糊
中值滤波	邻域中位数	**椒盐噪声**（脉冲噪声）	慢（要排序）、对高斯噪声一般
双边滤波	空间距离 × **灰度差**加权	保边去噪	慢（非可分离）、对强噪声无解
引导滤波	用引导图（如原图）做局部线性模型	保边、可做**边缘保持上采样**	需要引导图
非局部均值（NLM）	按图像块相似度加权	强去噪、纹理保留好	很慢

面试加分：**为什么高斯核可分离？** → $G(x,y) = G(x)\cdot G(y)$，二维卷积可拆成两次一维卷积，计算量从 K^2 降到 $2K$（K=7 时从 49 次乘加到 14 次）。

3σ 原则：高斯核半径通常取 3σ，超出部分的权重 < 0.3%，可以截断。

C4 ★★★ Canny 边缘检测的四步是什么？为什么需要非极大值抑制？

A：四步：

1. **高斯平滑**（去噪，避免噪声被误判成边缘）；
2. **计算梯度**（Sobel 算 gx、gy → 幅值 $\sqrt{(gx^2+gy^2)}$ 和方向 $\text{atan2}(gy,gx)$）；
3. **非极大值抑制（NMS）**：沿**梯度方向**看相邻两个像素，只有当前点幅值最大才保留 → 把"粗边缘"细化成**单像素宽**；

4. **双阈值 + 滞后连接**：高阈值确定"强边缘"，低阈值给出"弱边缘"候选，只保留与强边缘**连通**的弱边缘 → 抑制孤立噪声点，同时保住边缘的连续性。

为什么需要 NMS：梯度幅值在边缘附近是一个"脊"（几个像素宽都很大），不抑制就会得到粗边缘、后续做轮廓/直线拟合会重复计数。

追问：双阈值的两个值怎么定？→ 经验是高:低 ≈ 2:1 或 3:1；也有自适应方法（按梯度幅值分位数）。

C5 ★★ 直方图均衡化 / CLAHE 在做什么？为什么工业上常用 CLAHE?

A：**直方图均衡**把灰度分布映射成近似均匀分布（累积分布函数 CDF 作为映射函数），从而**提升对比度**。

问题：全局均衡会**过度放大平坦区域的噪声**（比如大片天空的细微噪声被拉到可见），且图中有强亮/暗区时局部仍然看不清。

CLAHE（限制对比度的自适应直方图均衡）：把图分块（如 8×8）各自均衡，并用 `clipLimit` **截断直方图的峰值**（超出部分均匀分回），再做块间**双线性插值**消除块边界。

为什么工业界爱用：夜间/逆光/雨雾场景下能显著提升视觉可见性，且参数少、可复现、无学习成本。**我们做低光照预处理时它就是首选基线**。

C6 ★★ 相机畸变怎么建模和校正?

A：两类畸变：

```
径向畸变（镜头形状）：  x_d = x(1 + k1·r² + k2·r⁴ + k3·r⁶),   r² = x²+y²
切向畸变（装配不平行）：  x_d = x + [2·p1·x·y + p2·(r²+2x²)],  y 方向对称
```

- k1 < 0 通常是**桶形**（广角/鱼眼），k1 > 0 是**枕形**。
- 校正：用内参 K（fx, fy, cx, cy）+ 畸变系数（k1,k2,p1,p2,k3），通过 `undistort` 反算 → 实际实现是**反向映射 + 插值**（因为正向映射会有空洞）。
- **鱼眼**要用专门的模型（等距投影 `r = f·θ`），普通针孔模型拟合不了。

面试加分：畸变系数怎么标定？→ 张正友标定法（多张棋盘格，用单应性求解内参初值，再非线性优化最小化重投影误差）；评估标准是**重投影误差**（RMS，好的标定 < 0.5 px）。

C7 ★★ 仿射变换和透视变换的区别？怎么求变换矩阵?

A：

	仿射（Affine）	**透视（Homography）**
自由度	**6**（保持平行性）	**8**（平行线可交于消失点）
矩阵	2×3（最后一行 [0,0,1]）	3×3（差一个尺度，8 自由度）
需要点对	**3 对**不共线点	**4 对**（任意三点不共线）
用途	旋转/缩放/平移/剪切	车道线 BEV、文档矫正、平面单应

求解：把对应点代入方程得到线性方程组，用 **DLT（直接线性变换）** 求最小二乘解（4 对点以上用 SVD），再用 RANSAC 剔除外点。

追问：为什么 BEV 变换常用单应而不是仿射？→ 因为相机是斜视地面的（透视投影），仿射保持不了"近大远小"，只有地面**平面**的单应才是精确的——**这也是为什么单应只对地面平面前提成立**（非平面区域会被拉变形）。

C8 ★★ 图像金字塔有什么用？拉普拉斯金字塔在做什么?

A： - **高斯金字塔**：逐层高斯模糊 + 下采样（每层尺寸减半），用于多尺度检测、特征匹配（SIFT 的尺度空间）。 - **拉普拉斯金字塔**：`L_i = G_i − upsample(G_{i+1})`，即"本层减去上层的上采样"，保存的是**各频段的细节**。 - **用途**：图像融合（把两张图的不同频段拼接，如多曝光融合/多聚焦融合）、图像压缩、超分。

为什么有用：不同尺度关注的语义不同（低分辨率看整体、高分辨率看细节），检测里的 FPN、分割里的多尺度特征都是这个思想的延伸。

C9 ★★ PSNR / SSIM / LPIPS 分别在衡量什么?

A:

```
MSE  = (1/N)Σ(I − K)²
PSNR = 10 · log10(MAX² / MSE)        (dB，越大越好；但与人眼感知相关性弱)
SSIM = (2μxμy + C1)(2σxy + C2) / [(μx²+μy²+C1)(σx²+σy²+C2)]    (结构相似性，[−1,1])
```

- **PSNR** 只看逐像素误差，对"整体亮度偏移"极敏感、对"结构错乱"不敏感 → 常出现 PSNR 高但肉眼更差的情况。
- **SSIM** 从**亮度、对比度、结构**三项比较，更贴近人眼（局部窗口计算后取均值）。
- **LPIPS** 用预训练网络（VGG/AlexNet）的**深层特征距离**，与人类判断相关性最高，是现在超分/生成的默认指标。

面试话术："指标要和任务匹配：去噪看 PSNR/SSIM，超分/生成看 LPIPS + 感知质量，检测分割还是看任务的 IoU/mAP。"

C10 ★ 形态学操作有哪些？各自的作用?

A：四个基本操作：

操作	定义	作用
腐蚀	结构元覆盖区域内**取最小**	缩小前景、断开细连接、去小噪点
膨胀	取**最大**	扩大前景、填补小孔
开运算	先腐蚀后膨胀	**去小噪点/毛刺**，面积不变
闭运算	先膨胀后腐蚀	**填小孔/缝隙**，面积不变

进阶：顶帽（`原图 − 开运算`，提取小亮目标）、黑帽（`闭运算 − 原图`，提取小暗目标）、形态学梯度（`膨胀 − 腐蚀`，粗边缘）。

用途：二值 mask 的后处理（去孤立点、填洞）、OCR 字符粘连分离、工业缺陷检测的预处理。

C11 ★★ 图像数据增广有哪些？哪些任务不能用?

A：按类型：

- **几何**：翻转、随机裁剪、旋转、缩放、仿射/透视扰动。
- **颜色**：亮度/对比度/饱和度抖动、色调偏移、gamma、JPEG 压缩、模糊、噪声。
- **遮挡/混合**：Random Erasing、Cutout、**Mixup**（两张图线性混合 + 标签混合）、**CutMix**（贴一块区域 + 标签按面积混合）。
- **策略搜索**：AutoAugment、**RandAugment**（随机选 N 种变换 + 统一强度 M，超参只有两个）。
- **多尺度**：随机分辨率训练（尺度抖动）。

哪些不能用（面试高频）：- **OCR/文档**：颜色抖动、翻转、强裁剪都会破坏语义（文字方向、颜色对含义有影响）。- **检测/分割**：裁剪后必须**同步变换标签**（框/掩码），否则标签错位。- **VLM 指令微调**：增广会让"描述内容"与图不一致（颜色变了但 caption 没变），**制造假监督**。- **医学/工业**：翻转可能违反解剖/工艺先验（左右心、螺纹方向）。

C12 ★ 频域方法（傅里叶/小波）在图像里有什么用?

A：三条实际用途：

1. **滤波设计**：频域里做理想/巴特沃斯低通（但会振铃），高斯低通无振铃（因为高斯是自傅里叶变换）。
2. **周期性噪声去除**：在频谱图上找到亮点（周期噪声的频点）挖掉，再逆变换——纺织/扫描件的经典手法。
3. **小波**：多分辨率分解（低频近似 + 高频细节），用于图像压缩（JPEG2000）、去噪（小波阈值）、医学融合。

深度学习里的对应：频域卷积（FFT 加速大核卷积）、频域域适应（把低频（风格）与高频（结构）分离做风格迁移/域泛化）。

C2 CNN 组件与计算（C13–C26）

C13 ★★★ 卷积的输出尺寸怎么算？给一个具体例子。

A：

```
O = floor( (I + 2P − K) / S ) + 1
```

（I 输入边长、P padding、K 核大小、S 步长）

带空洞卷积时：`K_eff = D·(K−1) + 1`，再代入上式。

例子（面试常现场问）：- 输入 224，K=7，S=2，P=3 → `(224 + 6 − 7)/2 + 1 = 112`（这正是 ResNet 的 stem：7×7/s2 → 再 maxpool 3×3/s2 → 56） - 输入 56，K=3，S=2，P=1 → `(56 + 2 − 3)/2 + 1 = 28`（降一半） - 输入 32，K=3，S=1，P=0 → `30`（**same 尺寸必须 P=(K−1)/2，K 为奇数**）

追问：为什么核大小通常是奇数？→ ① 有中心像素（便于定义"以谁为中心"）；② same padding 时 `P=(K−1)/2` 是整数。

C14 ★★★ 感受野怎么算？三个 3×3 堆叠等于一个 7×7 吗？

A：递推公式（从浅到深）：

```
RF_l = RF_{l−1} + (K_l − 1) · Π_{i<l} S_i        初值 RF_0 = 1
```

三个 3×3（stride 1）：`1 → 1+2=3 → 3+2·1=5 → 5+2·1=7` → **感受野 7，等于一个 7×7**。

但不等价于一个 7×7 卷积：

	3 个 3×3	1 个 7×7
感受野	7	7
参数	$3·(9C^2) = 27C^2$	$49C^2$
非线性	**3 层**（表达能力更强）	1 层
计算量	$3·9HWC^2$	$49HWC^2$

结论："感受野相同，但堆叠版的**参数只有 55%、非线性多两层**——这就是 VGG 全用 3×3 的原因。"

追问：有效感受野（ERF）是什么？→ 理论感受野内每个像素的贡献并不均等，实际"起作用"的区域是**高斯状**的，中心大、边缘小（可用梯度/扰动实验测）。**所以理论感受野 7 不代表边缘信息真被用上了**。

C15 ★★★ 参数量和 FLOPs 怎么算？举一个对比。

A：

```
标准卷积参数 = (Cin · K · K + 1) · Cout          (+1 是 bias)
标准卷积 FLOPs（近似 MACs）= H_out · W_out · Cin · K · K · Cout
```

具体例子（Cin=Cout=256, K=3, H=W=56）：

	参数	MACs
标准卷积	`(256·9+1)·256 = 589,824`	`56·56·256·9·256 ≈ 1.85 G`
深度可分离（DW+PW）	`256·9 + 256·256 = 67,840`（**约 1/8.7**）	`56·56·(256·9 + 256·256) ≈ 0.21 G`（**约 1/8.7**）

深度可分离卷积 = 逐通道卷积（每个通道一个 K×K，输出通道数不变）+ 逐点卷积（1×1，跨通道混合）：

```
参数 = Cin · K · K + Cin · Cout        压缩比 ≈ 1/Cout + 1/K²
```

面试提醒：报"参数量"和"FLOPs"要区分**是否含 bias、是否算乘加为 2 次**（有的论文算 2×MACs），否则数字对不上。

C16 ★★★ 1×1 卷积有什么用？为什么它能"降维"？

A：四个作用：

1. **跨通道信息融合**：本质是"在通道维度做全连接"（每个位置独立的线性组合）。
2. **降维/升维**：把 C 通道变成 C'，`参数 = C·C'`，极便宜。ResNet 的 bottleneck（256→64→64→256）就是靠它把计算量压到 1/4~1/8。
3. **改变通道数而不改空间尺寸**：投影到目标维度。
4. **加非线性**：1×1 conv + ReLU 等价于"逐位置的两层 MLP"。

为什么能降维：它不聚合邻域信息（K=1），只在通道方向做线性映射，所以是"通道上的矩阵乘"。

面试延伸：ViT 里的 patch embedding 也是用 `K=patch_size, S=patch_size` 的卷积实现的（等价于切块 + 线性投影）。

C17 ★★ 池化 vs 步长卷积，怎么选？

A：

	最大池化	平均池化	步长卷积
作用	保留最强响应、扩大感受野	平滑、保留整体能量	降采样 + **可学习**
优点	无参数、对位置小变化鲁棒	无参数、梯度均匀	学得到更好的下采样方式
缺点	丢位置信息、梯度只在最大处	弱响应被平均掉	有参数、可能引入棋盘/混叠

趋势：现代架构（ResNet-D、MobileNet、ConvNeXt）**更倾向步长卷积或 avgpool + 1×1**，因为 maxpool 丢掉的信息无法恢复；而"下采样要配合抗混叠（anti-aliasing）"（BlurPool）也是常考点。

C18 ★★★ BN / LN / IN / GN 的区别？CV 里为什么默认 BN？

A：

归一化	统计量的维度	依赖 batch?	场景
BN	每个通道，跨 (N, H, W)	**是**	CNN 标配（大 batch）
LN	每个样本，跨 (C, H, W)	否	Transformer / RNN
IN	每个样本每个通道，(H, W)	否	风格迁移（抹掉逐图风格差异）
GN	每个样本分成 G 组，组内 (C/G,H,W)	否	**小 batch 检测/分割**（BN 失效时的替代）

> BN: $y = \gamma \cdot (x - \mu_B)/\sqrt{(\sigma_B^2 + \varepsilon)} + \beta$　　训练用 batch 统计量，推理用滑动平均

CV 里默认 BN 的原因：卷积核共享 → 同一通道的统计量在空间上稳定，跨 batch 估计更可靠；同时 BN 平滑了损失曲面（允许更大学习率）。

BN 的坑（必答三个）：① **小 batch 时统计量噪声大**（检测/分割 batch=1~4 时掉点严重）；② **训练/推理不一致**（忘记切换 eval 模式 = 经典 bug）；③ **分布差异大的任务**（多域、风格差异）统计量不匹配 → 用 GN/LN 或 AdaBN。

C19 ★★★ 激活函数怎么演进？为什么 ReLU 取代了 sigmoid？

A：

激活	特点	问题
sigmoid / tanh	平滑、有界	**两端饱和 → 梯度消失**（导数最大仅 0.25）
ReLU `max(0,x)`	正区间梯度恒 1、计算极快、稀疏激活	**死亡 ReLU**（负区间梯度 0，学习率大时神经元永久失活）
LeakyReLU / PReLU	负区间给一个小斜率（可学习）	多一个超参
ELU / SELU	负区间平滑趋近 −1	含 exp，稍慢
GELU	`x·Φ(x)`（高斯 CDF 加权）	稍慢；Transformer/ViT 默认

激活	特点	问题
SiLU / Swish	`x · σ(x)`	主流大模型常用（平滑、非单调）

ReLU 取代 sigmoid 的三个原因：① 正区间**梯度不衰减**（解决深层网络梯度消失）；② 计算只有一次比较（比 exp 快得多）；③ 稀疏性（约一半神经元输出 0）→ 表征更解耦。

追问：SiLU/GELU 为什么更好？→ 在 0 附近**平滑且非单调**（有个小负值区），梯度更连续、对优化更友好；代价是计算量。

C20 ★★★ 残差连接为什么能缓解梯度消失？给个推导。

A：设残差块 `x_{l+1} = x_l + F(x_l)`，展开到第 L 层：

```
x_L = x_l + Σ_{i=l}^{L-1} F(x_i)
```

对 `x_l` 求导：

```
∂L_loss/∂x_l = ∂L_loss/∂x_L · ( 1 + ∂/∂x_l Σ F(x_i) )
```

关键在于那个 `1`：即使后面的 `∂F` 项因连乘而趋近 0，梯度仍能通过**恒等路径**无损回传。这是"梯度高速公路"。

附带好处：① 恒等映射让"加深至少不变差"成为可能（优化上的 ease）；② 特征复用/集成视角（ResNet 可看作多路径的隐式集成）。

追问：为什么是"加法"而不是"拼接"（DenseNet）？→ 加法**不增加通道数**（计算量恒定、可无限堆叠），拼接会增加通道数（后续层更贵，但特征复用更彻底）。

C21 ★★ 权重初始化怎么选？Xavier 和 He 的区别？

A：目标是**保持各层激活的方差稳定**（避免逐层放大/缩小）。

```
Xavier (Glorot) : Var(W) = 2/(n_in + n_out)      适合 tanh/sigmoid（对称激活）
He (Kaiming) :    Var(W) = 2/n_in                适合 ReLU（因为 ReLU 把一半激活置零，方差减半，所以要加倍）
```

推导思路：`Var(y) = n_in · Var(W) · Var(x)`，要 `Var(y)=Var(x)` 就得 `Var(W)=1/n_in`；ReLU 输出只保留一半方差 → 补一个 2 倍。

实践经验：① 现代框架的默认初始化基本可用；② **BN 存在时初始化就没那么敏感**（BN 会重新标准化）；③ 太小的初始化 → 信号逐层衰减；太大 → 爆炸。

C22 ★★ 反卷积（转置卷积）为什么会有棋盘格伪影？怎么避免？

A：转置卷积实现上采样时，输出像素被覆盖的次数**不均匀**（核重叠次数随位置变化），导致某些位置值偏大 → 网格状纹路。

四种避免方法：1. 用 `stride=1` 的卷积 + 前面的**插值上采样**（最常用、最稳，如 `upsample(bilinear) + conv3x3`）。 2. 用 `stride` 整除 `kernel`（如 S=2,K=4 或 S=2,K=2），让重叠均匀。 3. **Pixel Shuffle**（亚像素卷积）：先卷积出 $C \cdot r^2$ 通道，再 reshape 成高分辨率（无重叠问题，超分里常用）。 4. 用 resize + conv 替代（语义分割的常见做法）。

追问：转置卷积和"真反卷积"是一回事吗？→ **不是**。数学上的反卷积是卷积的逆运算；深度学习说的"反卷积"其实是**转置卷积**（梯度形式的卷积），不保证能还原输入。

C23 ★★ 空洞卷积的作用与副作用？怎么缓解？

A：**作用**：在不增加参数、不降分辨率的前提下**成倍扩大感受野**（`K_eff = D · (K−1)+1`），DeepLab 系列用它替代池化。

副作用（网格效应 / gridding）：连续堆叠相同 dilation 的空洞卷积时，**采样点只落在固定格点上**，中间像素永远不参与计算 → 小目标/细节信息被跳过。

缓解：① **混合 dilation**（如 1,2,5 交替，HDC 设计）；② 串联不同 dilation 的空洞空间金字塔（ASPP）；③ 与普通卷积/上采样混用。

面试加分："我们的多任务模型里用 ASPP 做粗尺度辅助头，本质就是用多个 dilation 覆盖不同感受野，让粗头的监督更全局——这个辅助头给了 0.2 的权重。"（**这条把 C23 和你的实际配置连起来了**）

C24 ★★ 为什么卷积有"平移等变性"，而它有局限？

A：卷积 + 池化带来**平移等变（translation equivariance）**：输入平移，输出跟着平移。这是极强先验，也是 CNN 数据效率高的原因。

局限（三个）： 1. **下采样破坏等变性**：stride/pooling 会让等变只在采样格点上成立（亚像素平移不保持），这就是"CNN 对小平移不够鲁棒"的原因之一。 2. **边界效应**：padding 让边缘位置的等变性不成立。 3. **全局等变 vs 位置敏感**：检测里"同一物体在不同位置应有不同输出"，所以需要位置编码/坐标监督来**打破**纯等变。

顺带：ViT 因为几乎没有这个先验，才需要海量数据；ConvNeXt 用大核 + LN + GELU 把架构现代化，证明"CNN 的归纳偏置仍有效"。

C25 ★★ 混合精度、channels_last、融合算子各优化什么？

A：

技术	优化什么	注意
AMP（fp16/bf16）	显存减半、TensorCore 提速	fp16 要 loss scaling（防下溢）；**bf16 不需要**（动态范围大，但精度低）
channels_last（NHWC）	访存连续性（GPU 上 NHWC 对 1×1 卷积/BN 更快）	与某些算子不兼容，需要全链路统一
融合算子（conv+BN+ReLU 折叠）	减少 kernel launch 与访存	推理时常见；**训练时 BN 统计量要小心**
梯度检查点	显存换时间（重算激活）	约 30% 时间代价
torch.compile / TensorRT	图优化 + 算子融合	动态 shape 支持有限

C26 ★★ 卷积和全连接的关系？为什么现代网络几乎不用大 FC 层？

A：**全连接是卷积的特例**（K = 输入尺寸、感受野覆盖全图）；反过来，把 FC 的权重 reshape 就是卷积核。

不用大 FC 的原因：① 参数量爆炸（`H·W·C → N` 的 FC 是灾难级参数，如 VGG 第一个 FC 就有 1 亿参数）；② 丢失空间结构（不做归纳偏置）；③ 不能处理变尺寸输入。

现代做法：**全局平均池化（GAP）+ 一层小 FC**（ResNet 的做法），参数量从亿级降到 `C·N`。

C3 经典架构演进（C27–C36）

C27 ★★★ 串一遍经典 CNN 的演进线，每一步解决了什么？

A：

模型	年份	关键贡献	解决的痛点
LeNet-5	1998	conv + pool + FC 的雏形	手写数字识别
AlexNet	2012	ReLU、Dropout、GPU 训练、数据增广	深网络能训起来了（sigmoid 饱和被 ReLU 解决）
VGG	2014	**全 3×3 堆叠**、模块化	证明"深度 + 小核"有效（参数量爆炸是代价）
GoogLeNet/Inception	2014	**多尺度并行分支 + 1×1 降维 + GAP**	用宽度换深度、把参数从亿级降到 500 万
ResNet	2015	**残差连接 + BN + bottleneck**	解决深层退化（56 层比 20 层差）与梯度消失
DenseNet	2017	密集连接（每层接收前面所有层）	特征复用、参数更少；代价是显存
MobileNet/ShuffleNet	2017-18	深度可分离、通道混洗	端侧算力约束

模型	年份	关键贡献	解决的痛点
EfficientNet	2019	**复合缩放**（深度/宽度/分辨率联合）	缩放策略从"拍脑袋"变"可搜索"
ConvNeXt	2022	把 Transformer 的设计（大核、LN、GELU、倒瓶颈）搬回 CNN	证明 CNN 的归纳偏置仍有价值

一句话串起来："AlexNet 证明能训深，VGG 证明深有效，Inception 证明要宽且省，ResNet 证明还能更深，后面的都在**效率**上做文章，ConvNeXt 是回头看'CNN 到底输在哪'。"

C28 ★★ Inception 模块的核心思想？为什么必须用 1×1 卷积？

A：**多尺度并行**：同一层输入同时过 1×1、3×3、5×5、maxpool 四条支路，再在通道维拼接 → 一个模块同时看不同感受野。

为什么必须 1×1 降维：如果直接对 256 通道做 5×5 卷积，参数是 $25 \cdot 256 \cdot 256 \approx 1.6M$；先用 1×1 把 256 降到 32 再 5×5，参数 $256 \cdot 32 + 25 \cdot 32 \cdot 32 \approx 34k$ ——**降了近 50 倍**。没有 1×1，这个模块根本装不下。

追问：Inception 和"多分支 ResNet（ResNeXt）"的区别？→ Inception 是**手工设计**的分支拓扑，ResNeXt 把分支变成**同构分组卷积**（组数可调），更规整、更好扩展。

C29 ★★★ ResNet 的 bottleneck 为什么是 1×1 → 3×3 → 1×1？

A：以 256→256 为例：

方案	参数量
直接 3×3：$9 \cdot 256 \cdot 256$	**589,824**
bottleneck：$1\times1\ 256\to64$ + $3\times3\ 64\to64$ + $1\times1\ 64\to256$	$16{,}384 + 36{,}864 + 16{,}384 = 69{,}632$（**约 1/8.5**）

逻辑：先用 1×1 **压缩通道**（降维）→ 在低维空间做 3×3（省算力）→ 再 1×1 **恢复通道**（升维）。这就是"bottleneck"命名的由来（中间窄、两头宽）。

深层设计原因：ResNet-50 之前用 basic block（两个 3×3），到 50 层以上必须用 bottleneck，否则算力下不来。

追问：为什么中间是 3×3 而不是更大的核？→ 3×3 是"最小的能同时感知上下左右"的核，且堆叠便宜（见 C14），是精度/效率的最优点。

C30 ★★ DenseNet 和 ResNet 的本质区别？

A：

	ResNet	DenseNet
连接	$x_{l+1} = x_l + F(x_l)$（**相加**）	$x_{l+1} = concat(x_l, F(x_l))$（**拼接**）
通道数	不变（可无限堆叠）	**线性增长**（第 l 层的输入通道 = $k_0 + k \cdot l$）
参数	普通	**更少**（因为每层只需输出很小的 k=32 通道）
显存	较低	**很高**（所有中间特征都要保存，供后续拼接）
特征复用	隐式（多路径集成视角）	**显式**（每层直接看到原始输入）

面试话术："ResNet 是加法式的恒等路径（省计算、能很深），DenseNet 是拼接式的全连接复用（参数效率高、但显存爆炸）。工业界主流是 ResNet 系，因为它好部署。"

C31 ★★★ 轻量化网络的三条路线？

A：

1. **减少卷积计算量**： - **MobileNet v1**：深度可分离卷积（DW + PW），算力降到约 1/8~1/9（见 C15）。 - **ShuffleNet**：分组卷积（group conv）省算力，但组间信息不流通 → **channel shuffle**（把通道打乱重排）恢复通信。 - **MobileNet v2**：**倒残差（inverted residual）**——先 1×1 升维 6 倍 → DW 3×3 → 1×1 降维，配 **线性瓶颈**（最后一层不用 ReLU，防止低维信息被截断）。

2. **减少冗余分支/结构搜索**：NAS（MnasNet）、EfficientNet（复合缩放：深度 ∝ α^ϕ、宽度 ∝ β^ϕ、分辨率 ∝ γ^ϕ，约束 $\alpha\beta^2\gamma^2 \approx 2$）。
3. **蒸馏/量化/剪枝**：结构不变，靠训练与压缩技术。

面试加分："端侧我会先算三件事——**参数（内存）、MACs（算力）、访存（带宽）**。很多轻量模型参数量很小但仍快不起来，原因是逐层访存（depthwise 的访存效率低、GPU 上不友好）。"

C32 ★★★ SE 和 CBAM 各是什么？SE 为什么能提点还不贵？

A：

- **SE（Squeeze-and-Excitation）**：`GAP`（把 H×W 压成 1×1）→ `FC → ReLU → FC → sigmoid` 得到每通道一个权重 → **逐通道缩放**特征图。本质是**通道注意力**。
- **CBAM**：**通道注意力 + 空间注意力**串联（通道分支用 avg+max 双池化，空间分支用一个 7×7 卷积生成 H×W 的权重图）。

SE 便宜的原因：两个 FC 之间有降维比 r（通常 16），所以参数量只有 $2 \cdot C^2/r$（C=256, r=16 → 8k），相比卷积的 590k 可以忽略。而收益是稳定的（SE-ResNet50 在 ImageNet top-1 上约 +1 个点量级）。

追问：SE 和"注意力机制"（Transformer 的 attention）是一回事吗？→ 不同。SE 是**通道级重新加权**（软特征选择），没有 query-key 的成对建模；Transformer 的 attention 是**位置间的动态聚合**。**但思想的共同点是"用全局上下文来调制局部响应"**。

C33 ★★ FPN 为什么要"自顶向下 + 横向连接"？

A：目标检测的核心矛盾：**深层特征语义强但分辨率低**（定位粗），**浅层特征分辨率高但语义弱**（分类差）。FPN 用两条路把它们结合起来：

1. **自底向上**（backbone 的正常前向）：得到 P3/P4/P5 不同尺度的特征；
2. **横向连接**：每个层级用 1×1 卷积把通道数统一（如 256）；
3. **自顶向下**：把深层特征**上采样**后与同层横向特征**相加** → 浅层也获得强语义。

结果：每个尺度都有"高语义 + 高分辨率"，小目标检测显著提升。**这也是分割（DeepLab/UNet）共用的思想。**

C34 ★★★ 分割网络的三条技术路线（UNet / DeepLab / HRNet）？

A：

路线	代表	核心	强项	弱项
编码-解码 + 跳连	**UNet**、SegNet	下采样提语义、上采样恢复分辨率，**skip 把浅层细节直接送回去**	医学/小数据、边界精细	参数量大（UNet 对称结构）
空洞卷积保分辨率	**DeepLab v1/v2/v3+**	不做激进下采样，用空洞扩大感受野；**ASPP**（多 dilation 并行 + 全局池化）捕捉多尺度	大场景语义分割	高分辨率下显存/算力大、空洞网格效应
全程高分辨率	**HRNet**	不做"先降后升"，多分辨率分支**并行 + 反复融合**	关键点/姿态/精细分割	计算量高

面试一句话："UNet 靠跳连找回细节，DeepLab 靠空洞不丢分辨率，HRNet 干脆全程不降——本质都是在解决'语义与分辨率的矛盾'。"

追问：ASPP 和 SPP 什么关系？→ ASPP 是 SPP 的空洞版（用多个 dilation 的并行卷积替代多尺度池化），都为了多尺度上下文。

C35 ★★★ 卷积和 Transformer 的本质取舍？ConvNeXt 做了什么？

A：

	CNN	ViT
归纳偏置	**强**（局部性、平移等变、权重共享）	**弱**（几乎无先验）
数据效率	小数据也能训	**需要大数据/强增广/蒸馏**
长程依赖	需要堆叠扩大感受野	**每层全局**
计算	$O(HWC^2K^2)$，与分辨率线性	$O(N^2)$，与分辨率平方
高分辨率	友好	贵（需窗口/稀疏注意力）

ConvNeXt 的结论：把 ViT 的现代设计搬回纯 CNN（**7×7 depthwise 大核、LN 替代 BN、GELU、倒瓶颈、更少的激活/归一化层**、LayerScale），就能追平 Swin Transformer —— 说明**当年"Transformer 更强"有一部分其实是"训练技巧与结构细节"的差异**，不全是自注意力的功劳。

面试话术："我会按落地约束选：数据少/要端侧 → CNN 或混合；数据多/要长程建模 → ViT；实际大都是混合（CNN 提局部 + attention 提全局）。"

C36 ★ 多模态模型为什么偏爱 ViT，但这个选择有什么代价？

A：**偏爱 ViT 的原因**：① patch token 序列**天然适配 LLM 的 token 接口**（不需要额外的空间池化/展平策略）；② 全局注意力对"整图关系"建模更强（对描述/推理类任务友好）；③ 与 CLIP 系预训练权重配套（直接可用）。

代价（这是加分点）： 1. **密集预测弱**：ViT 的全局注意力会把空间细节"抹匀"，做分割/深度/占用时要额外加 positional 信息或多尺度结构——**这正是我做 3D 占用时的体会：纯 ViT 特征对细粒度几何不够，需要 FPN/多尺度头来补**。 2. **高分辨率贵**：$O(N^2)$，所以要靠窗口注意力或 token 压缩。 3. **小数据难训**：必须依赖大规模图文预训练。

C4 训练与调优（C37–C44）

C37 ★★★ 学习率策略有哪些？为什么需要 warmup？

A：四种主流：

策略	形状	适用
Step / MultiStep	阶梯下降（每 N 轮 ×0.1）	经典 CNN（ResNet 的标准配方）
Cosine 退火	余弦曲线平滑降到 0	现代默认，配合 warmup
One-Cycle	先线性升到峰值再降到很低	训练轮数固定时收敛最快
Plateau	指标不涨就降	不确定总轮数时

为什么需要 warmup：训练初期参数随机、梯度方向噪声大（尤其 BN 统计量还没稳定），直接用大学习率会导致**震荡甚至发散**；warmup 让学习率从很小逐步升到目标值，给 BN 统计量和早期表征一个"稳定期"。Transformer/ViT 尤其依赖 warmup。

追问：余弦退火为什么比 step 好？→ 平滑下降避免"降学习率时的指标抖动"，且后期小学习率有利于精细收敛；另外 cosine 的周期性重启（SGDR）能帮助跳出局部极小。

C38 ★★★ SGD 和 Adam 怎么选？为什么经典 CV 模型用 SGD？

A：

	SGD + Momentum	Adam / AdamW
更新	用一阶动量平滑梯度，**所有参数同一学习率**	一阶 + 二阶矩，**逐参数自适应步长**
收敛速度	慢（要精调 lr 与 schedule）	**快**（对 lr 不敏感、易上手）
泛化	通常**更好**（尤其中小数据、CNN）	常常略差（有"尖锐极小"倾向）
显存	小（只存动量）	大（存 m、v，约 3 倍参数显存）

经典 CV 用 SGD 的原因：① 大量实验表明同样的超参预算下 SGD+动量 的**最终泛化**更好；② Adam 的自适应步长在卷积网络上容易收敛到"尖锐"的解；③ 传统 pipeline（长训练 + cosine + 大 batch）已经和 SGD 调得很顺。

但现在：Transformer/ViT/大模型基本都用 **AdamW**（解耦 weight decay），因为 LN + 自适应优化更匹配。

C39 ★★★ AdamW 和 Adam 的区别？为什么 weight decay 要"解耦"？

A：**Adam 的实现里，L2 正则被加进梯度**，然后被自适应学习率缩放：$\theta \leftarrow \theta - lr \cdot (\hat{g} + \lambda\theta)/\sqrt{\hat{v}}$。结果是**梯度大的参数被正则得弱、梯度小的被正则得强**（因为除以 $\sqrt{v}$），与"均匀惩罚大权重"的初衷不符。

AdamW 解耦：$\theta \leftarrow \theta - lr \cdot \hat{g}/\sqrt{\hat{v}} - lr \cdot \lambda \cdot \theta$，正则项**不经过二阶矩缩放**，直接按 lr 衰减权重。

效果：同样的 λ 下正则更有效、更稳定；这是现在大模型训练的标准做法。

面试延伸：这也是为什么"Adam 上直接调 weight_decay 经常没效果"——因为它被自适应缩放吃掉了。

C40 ★★ 正则化手段有哪些？分别在防什么？

A：

手段	机制	备注
weight decay / L2	惩罚大权重	与 BN 一起时要注意耦合（AdamW 解耦）
Dropout	随机置零 + 推理时缩放（或训练时 inverted dropout）	CNN 里现在少用（BN 已正则），FC 层仍常用
DropPath / Stochastic Depth	随机"跳过整个残差块"	**深网络/ViT 标配**，比 Dropout 更适合残差结构
Label Smoothing	软标签	防过度自信、提升校准
Mixup / CutMix	混合样本与标签	强增广，小数据提点明显；**训练更慢、对检测/分割需适配**
早停	验证指标不涨就停	最朴素也最有效
数据增广	见 C11	最"正统"的正则（不改模型）

追问：Dropout 和 BN 一起用为什么有时会掉点？→ 两者都在做"噪声/尺度"层面的正则，叠加后训练/推理的方差估计不一致（Dropout 改变激活方差，BN 统计量随之变化），实践中常改为只用其一。

C41 ★★ 迁移学习怎么微调？冻结哪些层？

A：按"数据量 + 领域差异"四象限决策：

	数据少	**数据多**
领域相近	只训分类头（冻结 backbone）	解冻全部，小学习率微调
领域差异大	训部分深层 + 头（浅层通用特征保留）	全部解冻（甚至从头训）

关键细节（面试常问）： - **BN 层**：微调时常见做法是**冻结 BN 的统计量**（保持预训练的 running mean/var，只训 γ/β），尤其小 batch 时——否则统计量会被目标任务的小 batch 带偏。 - **分层学习率**：浅层小（如 1/10）、深层大。 - **先冻后解冻**：先只训头几轮（稳定），再解冻微调（避免早期大梯度破坏预训练特征）。

C42 ★★ 训练 loss 正常但验证指标不涨，和过拟合怎么区分？

A：看**两条曲线的组合**：

现象	判断	处理
train loss ↓、val loss ↓ 但指标不涨	**指标与损失口径不一致**（见 S9 Q115）	拆指标成分、检查解码/阈值
train loss ↓、val loss ↑	**过拟合**	加正则/增广、减容量、早停
train loss 不降	**欠拟合或优化问题**	加容量、查 lr/初始化/数据管线
train loss ↓、val loss 平稳且指标差	**分布差异**（train/val 不同源）	查数据划分与预处理一致性

最常见也最隐蔽的一条：训练和推理的预处理不一致（归一化参数不同、resize 插值不同、通道顺序不同）——这种 bug 会让"训练很好、验证很差"，且极难从 loss 曲线看出。

C43 ★ 梯度累积和分布式训练的注意点？

A：

- **梯度累积**：`loss/N` 累加 N 步再更新，模拟大 batch。**注意 BN 的统计量仍按真实 batch 算**（不等价于真的大 batch，除非换 SyncBN 或 GN）。
- **DDP（DistributedDataParallel）**：多进程、每卡一个模型副本，梯度 all-reduce 同步——**推荐**。
- **DP（DataParallel）**：单进程多线程，主卡瓶颈严重——**已不推荐**。
- **SyncBN**：跨卡统计 BN，使"多卡小 batch"等价于"单卡大 batch"（检测/分割必备）。

C44 ★ 训练不稳定/掉点，排查顺序？

A：和 S9 Q117 同源，但 CV 特有两条**放在最前面**：

1. **数据管线一致性**（最常翻车）：训练/验证的 resize、归一化、通道顺序、色彩空间（BGR vs RGB）是否一致；
2. **BN 的 train/eval 模式**（忘记 `model.eval()` 是最经典 bug）；
3. 之后才是：label 越界 → 数值溢出（fp16）→ 学习率/warmup → 初始化/结构 → 硬件/框架。

C5 图像任务的评价指标（C45–C50）

C45 ★★★ 分类/检测/分割的核心指标各是什么？给一张能背的表。

A：

任务	主指标	定义要点	常见陷阱
分类	top-1 / top-5 acc	预测概率最高的类别是否命中 / 前 5 是否命中	类别不平衡时看 **macro-F1** 而非 acc
	AUC	排序质量，与阈值无关	极不平衡时 AUC 会虚高
检测	**mAP**	各 IoU 阈值下的 AP 平均	口径差异极大（见 C46）
	AP50 / AP75	IoU=0.5 / 0.75 的 AP	单一阈值不够全面
分割	**mIoU**	各类 IoU 的平均	小目标被大类淹没；边界不计
	Dice / F1	`2TP/(2TP+FP+FN)`	与 IoU 单调相关但不相等
	Boundary IoU	只算边界附近像素	对边界质量更敏感
深度	EPE / δ<1.25^k / bad pixel	平均端点误差 / 相对误差阈值 / 超阈值比例	尺度对齐方式不同

C46 ★★★ mAP 的口径差异有多大？VOC 和 COCO 怎么区分？

A：mAP 是"最容易被口径坑"的指标：

	VOC（2007/2012）	COCO
IoU 阈值	**单一 0.5**	**0.5 : 0.05 : 0.95 共 10 个**，再平均
PR 曲线积分	2007 用 11 点插值；2012 起用全点插值	全点插值（101 点采样）
类别平均	所有类 AP 的均值	同，但另有 APs/APm/APl（按面积）与 AR（召回）

结论：**COCO AP 的数值通常比 VOC mAP@0.5 低 15~25 个点**，两者不能直接比较。面试报指标必须说清"哪个口径、哪张表"。

追问：AP 的"插值"是什么？→ PR 曲线是锯齿状的，用"右侧最大值包络"（`p_interp(r) = max_{r'⩾r} p(r')`）消除锯齿后再积分。

C47 ★★ mIoU 和 Dice 什么关系？为什么分割常报 mIoU 又报 Dice？

A：两者单调相关，有闭式关系：

```
Dice = 2·|A∩B| / (|A| + |B|) = 2·IoU / (1 + IoU)
```

关系：IoU 0.5 ↔ Dice 0.667；IoU 0.8 ↔ Dice 0.889。所以 Dice 永远比 IoU 大。

为什么都报：① 领域习惯（医学爱用 Dice，自动驾驶爱用 mIoU）；② 两者对小目标的敏感度不同（Dice 对小目标更"宽容"，因为分母不含 TN）；③ 比赛/论文的口径要求。

Boundary IoU 补什么：mIoU 对"整体面积对、边界糊"不敏感；Boundary IoU 只在边界带内计算 IoU，专门衡量**边界质量**（这对占用/分割的精细度量很关键）。

C48 ★★ 深度/立体匹配的评价指标？

A：

- **EPE（End-Point Error）**：预测与 GT 的平均欧氏距离（视差或深度空间），最常用。
- **δ < 1.25^k（k=1,2,3）**：`max(pred/gt, gt/pred) < 1.25^k` 的像素比例（相对误差阈值），越大越好，是单目深度的标准指标。
- **Bad pixel（bad 1.0/2.0/3.0）**：误差超过阈值（如 1px 视差、或相对 3%）的像素占比。
- **AbsRel / RMSE / log10**：相对误差、均方根误差、对数误差。

坑：单目深度要看是否做过**尺度对齐**（median scaling / 最小二乘对齐）——不做对齐的 AbsRel 没有可比性（见 B 部分 Q108 尺度歧义）。

C49 ★ 生成质量的 FID/IS/CLIPScore 各有什么局限？

A：

指标	衡量	局限
IS	生成图的类别可分性	只在 ImageNet 类别上有意义、可被"每类各生成一张"刷高
FID	生成分布与真实分布在 Inception 特征空间的距离	依赖 Inception 特征（对纹理敏感）；**小样本时估计有偏**；对"语义错误但纹理像"不敏感
CLIPScore	图文一致性（CLIP 相似度）	只测粗粒度对齐（不擅长计数/空间关系）
人工评估	最终裁判	贵、主观、需要一致性设计

面试话术："FID 只能说明'像不像那个分布'，不能说明'对不对'——所以生成任务我都会配一个人工抽检或任务级指标。"

C50 ★ 图像任务的损失怎么选？（与 S9 交叉）

A：一句话汇总（详细见 S9）：

- **分类**：CE + label smoothing（不平衡用 focal）；
- **框回归**：SmoothL1 + IoU 族（GIoU/DIoU/CIoU）；
- **分割**：CE（主）+ Dice/Lovasz（辅）；
- **深度/视差**：SmoothL1 on 有效区域（+ 尺度不变项、+ photometric）；
- **去噪/超分**：L1（比 L2 更锐）+ 感知损失（LPIPS/VGG 特征）+ GAN 损失；
- **风格化**：内容损失 + 风格损失（Gram 矩阵）。

C6 与传统流程、多模态的接口（C51–C54）

C51 ★★ 预处理细节对 VLM/下游的影响有多大?

A：影响极大，且是最常见的"训练/推理不一致"来源。四个必须对齐的点：

1. **归一化参数**：CLIP 系用 `mean=(0.4815,0.4578,0.4082)`、`std=(0.2686,0.2613,0.2758)`（OpenAI 的口径），用 ImageNet 的 mean/std 会**直接掉点**。
2. **resize 策略**：短边 resize + 中心裁剪（分类）vs 直接拉伸（会变形，OCR 类任务致命）vs 保持长宽比 + pad。
3. **通道/色彩**：OpenCV 默认 **BGR**、PIL 是 RGB——混用会让红蓝互换（模型仍能跑，但性能莫名下降）。
4. **插值方式**：双线性 vs 双三次 vs area，在缩小倍数大时差异明显。

面试话术："我会把预处理写成一个**唯一函数**被训练和推理共用，并加一条断言校验（形状、dtype、均值范围），避免两边各写一套。"

C52 ★★ 图像增广在 VLM 微调里为什么有风险?

A：因为**标签是自然语言**，增广会破坏"图文一致性"：

- 颜色抖动 → 图像从"红色苹果"变"橙色苹果"，但 caption 还说红色 → **制造假监督**；
- 裁剪 → 目标被裁掉，但回答还在描述它；
- 翻转 → 文字反了、左右关系错了（"左边的车"变"右边"）；
- Mixup → 语义变成"两个物体的叠加"，而文本没有这样的描述方式。

安全做法：只做**不改变语义**的增广（轻微的缩放/亮度/JPEG 压缩/小角度旋转），或者用**同步生成的修改后文本**（成本高）。这也是"VLM 的数据增广比 CV 保守得多"的原因。

C53 ★ 深度学习时代，传统图像处理还有用吗?

A：有用，而且在工业里**不可替代**。四个位置：

1. **预处理**：去畸变、白平衡、CLAHE、去噪（这些是几何/物理校正面，学习做不好也不该学）。
2. **几何校正**：标定、单应、极线校正（立体匹配必须先做）。
3. **后处理**：形态学去噪、连通域分析、轮廓拟合、NMS（很多是确定性算子，比神经网更可控）。
4. **数据生产**：标注工具、自动标注（用传统几何生成伪标签）、数据质量检查（模糊/过曝/重复检测）。

一句话："神经网络负责**语义理解**，传统方法负责**几何正确性和确定性约束**——两者的分工在工业落地里很清晰。"

C54 ★★ 一个完整工业视觉链路的典型顺序?

A：从传感器到输出的标准流程（面试画出来很加分）：

```
采集 → 时间同步/去畸变（标定）→ 色彩/亮度校正（CLAHE、白平衡、去噪）
     → 几何变换（resize / 切块 / 单应到 BEV）→ 归一化 → 网络推理
     → 后处理（解码、NMS、阈值、形态学）→ 跟踪/融合 → 业务规则
```

每一环都可能成为瓶颈：采集（曝光）、同步（时间戳）、校正（标定漂移）、预处理（CPU 耗时）、推理（prefill/decode）、后处理（NMS 在 CPU 上很慢）。

面试收尾："所以我判断一个 CV 系统的问题，习惯先看**链路哪一段的输入输出不对**，而不是直接怀疑模型——80% 的线上问题在模型外面。"

C7 视觉基础模型：DINO 家族与 DPT（C55–C68）

> 这一节补两个在面试里高频、但极容易被答混的东西：**DINO（有两个完全不同的算法同名）** 和 **DPT（ViT 做密集预测的经典改造）**。它们分别对应两条真实需求：**选视觉塔**（DINO/DINOv2）和 **做像素级输出**（DPT，深度/分割）。

C55 ★★★ DINO 有两个完全不同的算法，怎么区分？

A：这是面试的高频陷阱，**同名但毫无关系**：

	DINO（自监督）	**DINO（检测器）**
全称	**DI**stillation with **NO** labels	**D**ETR with **I**mproved de**NO**ising anchor boxes
出处	Meta / Inria，2021（Caron 等）	IDEA，2022（Zhang 等）
任务	**自监督预训练**（不需要标签，学表征）	**目标检测**（有监督，出框）
输出	视觉特征 / 注意力图	检测框 + 类别
关键词	teacher-student 自蒸馏、EMA、multi-crop、防坍塌	3D 可变形注意力、对比去噪、混合 query 选择
在 COCO 的地位	—	首个在 COCO 上刷到 63+ AP 的 DETR 系模型

答法："您说的 DINO 如果是视觉塔那个，是自监督的 DINO/DINOv2；如果是在检测榜单上那个，是 DETR 系的 DINO 检测器——两者只是缩写撞名了。"（**这一句能立刻分清你在哪一层**）

C56 ★★★ 自监督 DINO 怎么训练？为什么不会坍塌？

A：**自蒸馏（self-distillation）**：

```
同一张图做两个不同视角（crop/增广）→ student 看视角 A、teacher 看视角 B
teacher 的输出经过 centering + sharpening 后作为"软标签"
student 输出与之算交叉熵；teacher 的权重 = student 权重的 EMA（不参与梯度）
```

三个防坍塌的机制（必答）：

1. **EMA teacher**：teacher 更新慢（动量 0.99+），提供稳定的目标，避免 student 和自己同步跑偏。
2. **Centering**（减均值）：把 teacher 输出的均值减掉，**防止某一维主导**（否则所有样本都预测成同一个 one-hot）。
3. **Sharpening**（低温 softmax，$\tau \approx 0.04$）：让 teacher 分布变尖，**防止输出塌成均匀分布**。

关键理解：centering 和 sharpening 的作用**方向相反**（一个鼓励均匀、一个鼓励尖锐），两者**必须配对**——只做 centering 会塌成均匀，只做 sharpening 会塌成 one-hot。

与对比学习的区别：DINO **不需要负样本、不需要超大 batch**（SimCLR/MoCo 靠负样本防坍塌），这是它工程上更友好的原因。

C57 ★★ 多裁剪（multi-crop）策略起什么作用？

A：每张图取 **2 个 global crop（大视野，如 224）+ 若干个 local crop（小视野，如 96）**：

- **global crop 之间**互相比（学全局语义、物体级不变性）；
- **local crop 只和 global 比**（学局部到全局的对应，即"这块局部属于哪个整体"）。

收益：在不增加 batch 的情况下大幅增加"同一图像的视角数"，等价于廉价的强增广；而且逼模型学"局部—全局"关系（这也是 DINO 的 attention map 能自然对应物体区域的原因之一）。

追问：local crop 之间为什么不互相比？→ 两个小局部之间可能本来就不属于同一个物体（甚至没有重叠），强行对齐会引入错误监督。

C58 ★★★ DINOv2 相对 DINO 的关键改进？

A：五个层面：

层面	改进
数据	自建 **LVD-142M**（约 1.4 亿图文/图像）——用自监督检索 + 去重从公开数据里"挑"出与目标分布相似的子集（而不是直接堆量）
训练目标	自蒸馏 loss **+ 掩码图像建模（iBOT 式）** 双目标：前者学不变性，后者学局部细节
防坍塌	**Sinkhorn-Knopp centering**（替代简单减均值）+ **KoLeo 正则**（鼓励特征在空间里分散，防止挤成一团）
规模	从 ViT-S 一路到 **ViT-g（约 11 亿参数）**，且**蒸馏出小模型**（大模型当 teacher，小模型学）
使用方式	明确主打"**冻结特征直接可用**"：linear probe / k-NN 就接近微调效果

一句话："DINOv2 = DINO 的自蒸馏 + iBOT 的掩码建模 + 更大的精挑数据 + 更好的防坍塌与正则。"

C59 ★★ DINOv2 的特征和 CLIP 特征有什么本质差别?

A:

	CLIP	DINOv2
监督信号	**图文对**（语言监督）	**纯视觉自监督**（无语言）
特征倾向	**语义**（"这是什么"、可被语言描述）	**结构与实例**（形状、部件、同类区分）
优势任务	零样本分类、检索、图文对齐	分割、深度、对应、检测（**密集任务**）
弱点	细粒度空间与实例区分较弱	没有语言接口（不能做零样本文本查询）

为什么会这样：语言监督会把特征"压缩成语义类别"（猫的照片和猫的简笔画在 CLIP 空间接近）；而无语言监督必须靠**实例区分**（这只猫 vs 那只猫），所以保留了更多结构与个体信息。

面试加分："所以做 VLM 时，如果任务是'描述/问答'，用 CLIP 塔；如果要'精准定位/细粒度属性'，DINOv2 更合适；两者特征拼接（concat）也是常见做法——一个管语义、一个管结构。"

C60 ★★ 什么时候用 DINOv2 做视觉塔，什么时候用 CLIP?

A：决策表：

任务特征	选
图文检索、零样本分类、描述生成	**CLIP/SigLIP**（有语言对齐）
需要细粒度定位（grounding、OCR 边界、计数）	**DINOv2 打底**，或在 CLIP 塔上再加 DINOv2 分支
密集预测（分割/深度/占用）	**DINOv2**（结构信息丰富）
端侧/预算紧	CLIP 系成熟小模型（ViT-B）或蒸馏后的 DINOv2 小模型

坑（值得提）：DINOv2 的 patch 是 14，输入分辨率要求是 14 的倍数（如 224/336/518），不能随便 resize；而且它的特征维度（ViT-g 1536 维）比 CLIP 大，接 LLM 时投影层要更宽。

C61 ★★ 顺便问：DINO 检测器（同名那个）的四个关键技术?

A：简要四个（如果面试官问的是检测）：

1. **3D 可变形注意力**：参考点 + 采样偏移，只关注周围少量关键位置（比全局注意力省，且适合多尺度）。
2. **对比去噪（contrastive denoising）**：给 GT 加噪声生成正负 query（正：小噪声；负：大噪声），让模型学会"区分接近 GT 的框和干扰框"，加速收敛。
3. **混合 query 选择（mixed query selection）**：位置部分用 encoder 的 top-k 输出初始化，内容部分用可学习 query（因为 encoder 输出的内容信息不可靠）。
4. **look forward twice**：梯度回传时用上一层的梯度信息修正下一层的参数更新（"看两次"），提升精度。

一句话："DINO 检测器的三个关键词是**去噪、混合 query、两次前向**——它解决的是 DETR 收敛慢的问题。"

C62 ★★★ DPT 是什么？为什么 ViT 不能直接做密集预测？

A：**DPT（Dense Prediction Transformer）** 是把 ViT 改造成"能输出像素级结果"的经典架构（用于单目深度估计、语义分割）。

ViT 直接做不了密集预测的三个原因： 1. **只有单一低分辨率**：patch 16 的 384 输入只有 24×24 个 token，输出的"特征图"太小，无法恢复边界； 2. **没有多尺度特征**（不像 CNN 有 P3/P4/P5 的层级），而密集预测恰恰依赖多尺度； 3. **没有 skip 连接**：UNet 的精度来自"深层语义 + 浅层细节"，ViT 缺这一半。

DPT 的解法：从 ViT 的**多个中间层**取 token → **reassemble** 成空间特征图 → **逐级上采样 + 卷积融合**（把深层的全局语义与浅层的细节结合）→ 输出全分辨率预测。

一句话："DPT 就是给 ViT 补上 CNN 的'金字塔 + skip'这两件事，同时保留 ViT 的全局感受野。"

C63 ★★★ DPT 的 reassemble 和 fusion 具体做了什么？

A：

Reassemble（把 token 变回图像）：

```
输入：某一层 ViT 的 (N+1, D) token（含 CLS）
步骤：① 丢掉 CLS，把剩余 N 个 token reshape 成 (D, H/p, W/p)
     ② read：把相邻 2×2 个 token 在通道维拼起来（类似 pixel shuffle），空间减半、通道 ×4
     ③ project：1×1 卷积把通道投到目标维度（如 256）
     ④ 再双线性上采样到 patches 分辨率的 4 倍（即 H/4）
```

Fusion（逐级融合，从深到浅）：

```
从最深的 reassemble 特征开始：
  上采样 2× → 与下一层的 reassemble 特征 concat → 两层 3×3 卷积（通道减半）
  重复到最高分辨率
最后：Refine 模块（stride 2 下采样再上采样，抑制上采样的棋盘/伪影）→ 1×1 conv 输出 1 通道深度
```

为什么 reassemble 要"2×2 拼通道"：ViT 的 token 之间是**无空间邻接信息**的（只是一维序列），把 2×2 邻居拼进通道，等于**显式地把空间邻接关系还原**，比单纯 reshape 更能保住局部结构。

追问：为什么要用多个中间层而不是只用最后一层？→ 最后一层语义最强但空间细节最弱；**中间层（如第 6、9、12 层）兼有细节与语义**，这正是"ViT 版的 FPN"。

C64 ★★ 为什么 DPT 的深度边界比 CNN 更锐利？

A：两个原因：

1. **全局注意力**：每个 token 都能"看到"整幅图，所以它能判断"这块区域和远处那块是同一个平面还是不同物体"，而 CNN 受局部感受野限制、容易在边界处模糊。
2. **多尺度融合时的高分辨率通路**：reassemble 保留了 patch 级的空间信息，配合逐级上采样，边界不会被过度平滑。

代价：ViT 的 token 少（384 输入只有 576 个），细节恢复依赖上采样卷积——所以在"极细的边界"（如铁丝网、远处小物体）上反而不如高分辨率 CNN；这也是后来 Depth Anything 等用**更大输入分辨率 + 更强数据**来补的原因。

C65 ★★★ DPT 的训练损失为什么必须"尺度/位移不变"？

A：因为**单目深度有尺度歧义**：同一张图，预测 2 米和 4 米可能只差一个全局缩放，而"形状结构"完全正确。如果用普通 L1，模型会把大量梯度花在"整体缩放"这个本就不确定的方向上。

DPT 用 **scale-and-shift-invariant loss**：

```
先对预测做最小二乘对齐：d* = a · d_pred + b
其中 a、b 由最小二乘闭式解得到（与 GT 对齐）
然后 L = (1/n) Σ |d* − d_gt|
```

这正好对应 B 部分 Q108 的 SSI（尺度不变损失）——两者是同一个思想：**先消除不确定的全局尺度，再比较结构**。

追问：什么时候不能用尺度不变损失？→ 下游需要**绝对度量**时（占用、可通行区域、测距）。此时要靠传感器（激光/双目）提供绝对尺度，或用少量带尺度标注的数据做校准。

C66 ★★ DPT vs UNet vs SegFormer 的 MLP 头，怎么选?

A:

方案	结构	优势	劣势
UNet	CNN 编码-解码 + skip	边界精细、成熟、数据效率高	全局上下文弱（要靠深层堆叠）
DPT	ViT 多层 + reassemble + 融合	全局一致性好、边界锐利	参数量大、需要大输入分辨率才发挥
SegFormer 头	多尺度特征统一 MLP + 融合（无卷积）	极轻量、快	精细边界略弱

选型口诀："数据少、要精细边界 → UNet；要全局一致性（大面积深度/平面）→ DPT；要端侧速度 → 轻量 MLP 头。"

C67 ★★ Depth Anything 和 DPT 是什么关系?

A：**Depth Anything 用 DPT 作为解码头**，核心创新在**数据与训练策略**，不在网络：

1. **数据规模化**：用 6200 万张无标注图像做自监督（教师模型生成伪深度，学生学），相当于把 DPT 放进"数据引擎"里；
2. **语义先验**：利用 DINOv2 初始化视觉编码器（**注意：DINOv2 + DPT 是当前单目深度的主流组合**）；
3. **强增广**：对无标注数据做强扰动（颜色、模糊），逼模型对扰动不变；
4. **V2 的改进**：合成数据 + 更大教师 + 分阶段训练，并解决了"深度图有幻觉细节"的问题。

面试话术："Depth Anything 的贡献不是结构，而是把 DPT 这类头放进了'大规模数据 + 强增广 + 教师蒸馏'的流水线——这正好说明在视觉任务里，**数据和训练策略的收益常常大于结构创新**。"

C68 ★ 如果面试官说"我们想用 DINO 做视觉塔"，你会怎么回应?

A：这是个开放题，分三步答（展示你能落地）：

1. **先确认是哪个 DINO**："您指的是自监督的 DINO/DINOv2，还是检测的 DINO？"
2. **如果确定是 DINOv2 做视觉塔**，指出两个必须解决的对接问题： - **没有语言对齐**：DINOv2 特征不能直接零样本查询，需要靠指令微调把语义"教"进去（或与 CLIP 特征融合）； - **分辨率约束**：patch 14，输入必须是 14 的倍数；token 数与分辨率挂钩，要注意预算。
3. **给出建议**："我倾向 DINOv2 做**密集任务**（定位/深度/占用）的特征源，语义问答类仍用 CLIP/SigLIP；如果两者都要，就双塔拼接后过投影层——代价是 token 数和显存翻倍。"

这道题的价值：它同时考"你分不分得清同名模型"、"你知不知道 DINOv2 的局限"、"你能不能给出取舍"。

C8 Depth Anything V2 详解（C69–C78）

> Depth Anything 是当前**单目深度估计（MDE）的事实基线**，V2（2024.06）是零样本泛化的最强开源方案之一。 这一节把它的"为什么这么做"讲透——它是"数据策略胜过结构创新"的教科书案例。

C69 ★★★ V2 相对 V1 的核心改变是什么？（一句话能答）

A：**换掉了教师模型的训练数据来源**——V1 用"有标注真实图像"训教师，V2 改用"**合成图像**"训教师。

```
V1: 1.5M 有标注真实图 → 教师(ViT-L) → 给 62M 无标注真实图打伪标签 → 训学生
V2: 595K 合成图（Hypersim + Virtual KITTI 2）→ 教师(DINOv2-G) → 给 62M 无标注真实图打伪标签 → 训学生
                    ↑ 关键：教师换成"精确深度的合成数据"训练，规模也从 ViT-L 提到 ViT-G
```

为什么这个改动有用：伪标签的质量决定学生上限。真实数据集虽然有标注，但标注本身有噪声、覆盖窄、且**深度边界不精确**（标注来自传感器/立体匹配）；合成数据的深度是**渲染出来的真值，边界精确到像素**。用精确数据训教师，它给出的伪标签也就更干净。

C70 ★★★ 为什么"用合成数据训教师"反而更好？（最反直觉的一点）

A：三层理由：

1. **合成数据的深度是"精确的"，真实标注的深度是"近似的"**。室内数据集（如 NYU）的深度来自结构光/Kinect，边界处误差大；自动驾驶数据集来自激光，稀疏且需插值。而合成场景的深度由渲染管线直接给出，**每个像素都精确**。
2. **教师只需要"相对深度关系正确"，不需要覆盖所有场景**——它之后要在 62M 真实图上做预测，**分布多样性由被标注的数据提供，不需要教师见过**。这就是"精确性"比"多样性"更重要的原因。
3. **合成数据的分布窄是可以被接受甚至有益的**：场景简单、纹理干净 → 教师不会学到"纹理强的区域深度变化大"这种错误先验（V1 的幻觉细节正是这么来的）。

一句话："教师的任务是'给出精确的相对深度'，不是'认识所有场景'——所以用精确但窄的合成数据训，比用多样但带噪的真实标注训更合适。"

C71 ★★ 完整的三阶段训练流程？

A：

```
Stage 1｜教师微调
  DINOv2-G (ViT-g, ~1.1B) + DPT 头 → 在 595K 合成图上微调（编码器极小 lr，解码头较大 lr）
  → 得到一个"在合成域上精确"的教师

Stage 2｜伪标签生成
  用教师在 62M 无标注真实图上推理 → 得到伪深度
  → 质量筛选：去掉天空区域、去掉不可靠区域（透明/反光/极端光照）

Stage 3｜学生训练
  学生 = ViT-S/B/L + DPT 头 → 在 62M 伪标签图上训练（全参数）
  训练时用强增广（颜色抖动、高斯模糊、CutMix 等）逼学生学"与外观无关的几何" → 提升泛化
```

出三个尺寸的学生：ViT-S（24.8M）、ViT-B（97.5M）、ViT-L（335.3M）——**ViT-S 就已经接近 V1 的 ViT-L，但快约 10 倍**（这是最有说服力的工程收益）。

C72 ★★ 为什么教师要用 DINOv2-G（1.1B）？大教师对小学生的价值在哪？

A：两个层面的原因：

1. **DINOv2 的特征本身适合密集任务**（无语言监督 → 保留结构与实例信息，见 C59），适合做深度的几何基础。
2. **教师越大，伪标签越准**——论文观察到：即使大教师在**小合成集上已经过拟合**，它产出的伪标签仍然更好。原因很关键：**伪标签需要的是"相对深度关系正确"（谁近谁远），而不是合成域上的绝对精度**。大模型的表征能力强，在真实图上泛化出的深度关系更可靠。

面试话术："这是个很典型的'教师-学生规模解耦'案例——教师可以又大又会过拟合，只要它的伪标签在真实分布上关系正确；学生小、推理快，能力靠教师蒸馏过来。"

C73 ★★ 伪标签怎么筛选？为什么必须筛？

A：主要是**天空掩码 + 不可靠区域剔除**：

- **天空**：天空的深度在物理上无定义（无限远），标成任何值都是错的。V1/V2 都会把天空区域 mask 掉（或用语义分割/启发式检测）。
- **透明/反光物体**（玻璃、水面、镜子）：深度本身有歧义，容易产生假的深度跳变。
- **极端光照/过曝区域**：纹理信息缺失，模型只能靠语义猜。

为什么必须筛：伪标签是被当作"GT"用的，**错误伪标签会被学生当真理学进去**——这就是 V1 细节幻觉的根源。

C74 ★★★ V2 用的损失函数是什么？为什么必须尺度不变？

A：**scale-shift invariant loss（尺度-位移不变损失）**，与 V1 一致：

```
对"预测"和"GT"各自做同一套归一化（两边都要做，尺度/平移才被同时消除）：
    d^ = (d  − t(d))  / s(d)      t(d) = median(d),  s(d) = mean(|d − t(d)|)
```

```
    d* = (d* − t(d*)) / s(d*)
然后 L = (1/n) Σ |d^− d*|
```

三步走的实质：① 用**中位数**做平移对齐；② 用**平均绝对偏差**做尺度对齐；③ 在归一化空间里算 L1。 **为什么两个统计量选 median 和 MAE 而不是 mean/std**：中位数和平均绝对偏差对离群值（天空、极远点、遮挡空洞）更鲁棒，不会被少数极端深度带偏。

为什么必须：单目深度只有**相对**意义（仿射歧义）——预测 2 米和 4 米可能只差一个全局缩放。直接 L1 会让模型把梯度花在"整体缩放"这个本就不确定的方向上。

注意：这个损失**只在训练时**消除尺度；推理时输出的是"相对深度"，要用到度量任务必须另做尺度对齐（见 C77）。

C75 ★★ DA-2K 是什么？为什么要新造一个评测集？

A：**DA-2K** = 2000 张图，**人工标注的成对深度关系**（"A 比 B 更近还是更远"），用于评测相对深度的**边界/细节精度**。

为什么需要：现有 MDE 评测集有三个问题：① 深度 GT 本身有噪声（传感器/立体匹配），"接近 GT"不等于"对"；② 不同数据集用不同指标（AbsRel/RMSE/δ），口径不一致；③ **无法反映"细粒度边界是否准"**（V1 的幻觉细节在旧指标上分数很好）。

DA-2K 的做法：标注像素对之间的相对关系（人眼很容易判断），再用"判断正确的比例"作为指标。并在标注中结合了强 VLM 辅助（如 Qwen-VL / GPT-4o 先给初标、人工校验），提高了效率。

意义：这是"**指标比模型更容易骗人**"的正面案例——换成成对判断后，V1 的幻觉细节问题立刻暴露，V2 的提升也变得可量化。

DA-2K 的指标具体怎么算（这是一个纯粹的排序判断，没有"损失"）：

```
标注：一张图上标注若干像素对 (i, j)，标签是"哪个更近"（深度值更小 = 更近）
预测：模型输出深度图 d
判断：pred_label = (d[j] < d[i])  →  判为"j 更近"
指标：accuracy = 判断正确的对数 / 总对数
```

注意三点：① 它**只依赖相对顺序**，所以对全局尺度/平移完全免疫；② 它**不看绝对值**，所以度量错误、细节幻觉都会以"关系错"的形式暴露；③ DA-2K 特意在**边界/细结构等挑战区域加密标注**，否则随机像素对会被大片正确区域稀释。

我用一个 64×64 的合成例子实测了三个指标的行为（代码见 `demo_da2k_metrics.py`）：

情况	AbsRel	RMSE	δ<1.25	SSI 损失	成对准确率
A 完美预测	0.000	0.000	1.000	0.0000	1.000
B **只差一个仿射变换**（×2 + 3）	**2.558**	4.945	**0.000**	**0.0000**	**1.000**
C **细节幻觉**（局部区域关系反了）	**0.013**	0.161	**0.976**	0.1380	0.948

这张表把两件事一次说清了： - **B 行**：模型其实**完全正确**（只差全局尺度），但 AbsRel=2.56、δ=0 会把它判成"彻底失败" → 旧指标**冤枉**尺度歧义； - **C 行**：模型有细节幻觉，但 AbsRel=0.013、δ=0.976 **看起来非常好** → 旧指标**看不出**局部错误。

把 C 的成对准确率按区域拆开看，问题更清楚：

```
随机像素对   : 0.971   ← 大片区域是对的，把错误稀释了
边界/细节对 : 0.917   ← 细节幻觉在这里暴露
```

这就是 DA-2K 的意义：指标必须**对"关系错"敏感、对"尺度差"免疫**，才能测出 V1 那种"整体对、细节编"的问题。

C76 ★★ V1 的"细节幻觉"（hallucinated details）到底是什么问题？

A：V1 在以下场景会**凭空生成看起来合理但错误的深度细节**：

- **透明物体**（玻璃杯、窗户）：模型画出不存在的物体轮廓；
- **细结构**（毛发、铁丝网、细线）：出现假的深度跳变/条纹；
- **纹理丰富的平面**（草地、格子布）：把纹理误当成几何起伏；
- **天空/远景**：给出不一致的深度块。

根因：V1 的教师用**有标注真实数据 + 大量网络无标注图**训练，网络图像本身带有**贴纸、水印、纹理噪声**，教师从中学到了"纹理强 = 深度变化"的错误关联，再通过伪标签传给学生。

V2 怎么解决的：改用干净、精确的合成数据训教师 → 切断了这个错误关联的来源。**这也是"数据质量 > 数据规模"的直接证据**。

C77 ★★ Depth Anything V2 能不能直接用于度量任务（绝对深度）？

A：**原版不能**，它是仿射不变的相对深度。要用在度量任务上有三条路：

1. **Metric 版本**：官方发布的 `Depth-Anything-V2-Metric-Indoor / Outdoor`，在带绝对尺度的合成数据（Hypersim/VKITTI2 的 metric 标注）上微调，直接输出米制深度。**前提是场景类型匹配**（室内用它训室内、室外用室外），跨域会掉。
2. **用传感器标定尺度**：拿激光/双目/雷达的稀疏深度，对 DA-V2 的相对深度做 **scale-shift 对齐**（最小二乘解 a、b），再用于度量判断——**这是工业落地最常见、最稳的做法**。
3. **只用"关系"不用"数值"**：做排序、可通行区域（比阈值）、遮挡判断这类任务，相对深度就够。

面试话术（对你特别有用）："我会用 DA-V2 做稠密的几何先验、用激光/双目做尺度锚点，两者做一次最小二乘对齐——这样既有单目的稠密性，又有传感器的度量正确性。"

C78 ★ 在你自己的感知任务里怎么用它？（三条可落地）

A：

1. **补激光稀疏**：远距离/低线束区域激光点稀疏，DA-V2 提供稠密相对深度，用近处激光点拟合 scale-shift 后插值补远——**这是单目深度最主流的工程用法**。
2. **作为占用/可通行预测的几何先验**：把 DA-V2 的深度（或它的倒数 = 视差图）作为额外输入通道送进 BEV 网络；或用它做"自由空间的初始猜测"，让占用头只学修正量（类似残差预测）。
3. **训练期作为辅助监督**：给没有激光的场景提供深度自监督信号（配合光度一致性），缓解 GT 覆盖率不足的问题。

三个必须注意的坑（能说出来说明你真跑过）： - **逐帧推理会闪烁**（视频时序不一致），要加时序平滑或滑窗中值； - **DA-V2 在天空/透明物体上不可靠**，用作几何先验时要 mask 掉（V2 自己也 mask）； - **强增广训练出来的模型对"外观"不敏感，但对"极端光照"仍会崩**，夜间/雨雾要么单独训，要么只用它做近场。

C9 激活函数专题（C79–C88）

> 激活函数考的是"**为什么选它、它有什么病、怎么治**"。这一节把每条都给出实测数字或具体机制，不只列公式。

C79 ★★★ 为什么必须有非线性激活？全用线性会怎样？

A：因为**多层线性叠加仍等价于一层线性**：

> $W_2(W_1x) = (W_2W_1)x = Wx$　　← 三个矩阵乘等于一个矩阵乘

所以没有激活函数，无论堆多少层，模型的表达能力都只有"一个线性变换"——**连 XOR 都学不会**（XOR 不可线性分割）。

一句话：激活函数是深度的**唯一意义来源**；它把网络从"线性回归"变成"通用函数逼近器"。

追问：神经网络为什么能逼近任意连续函数？→ 万能逼近定理（Universal Approximation）：**单隐层 + 非线性激活 + 足够宽的神经元**可以以任意精度逼近紧集上的连续函数。注意它只说"存在"，不说"能训出来"——所以实际要靠深度 + 优化技巧。

C80 ★★★ sigmoid 为什么被淘汰？（三个具体原因 + 一个实测数字）

A：

原因	说明
① **梯度消失（最致命）**	导数 $\sigma'(x) = \sigma(1-\sigma) \leqslant 0.25$。**实测最大导数正好 0.25**。10 层连乘就是 $0.25^{10} \approx 1e\text{-}6$——梯度彻底消失。**tanh 的导数上界是 1.0、ReLU 是 1.0**，所以活下来了。

原因	说明
② **输出非零中心**	`σ(x) > 0` 恒成立，导致下一层的输入全为正 → 梯度更新方向被"锁"在同一个象限 → 出现 zigzag 震荡，收敛慢。tanh 是零中心的，所以在 RNN/生成模型里仍有一席之地。
③ **计算贵**	含 `exp`。实测 1e6 个元素：sigmoid 5.9ms vs ReLU 3.2ms（差约 2 倍，在 NPU 上差距更大）。

注意：sigmoid **没有消失**，它还有两个正当用途：**二分类输出层**（概率意义）、**门控机制**（LSTM 的遗忘门、GLU 的门）。

C81 ★★★ ReLU 好在哪？有什么问题？

A：

```
ReLU(x) = max(0, x)        导数 = 1 (x>0) 或 0 (x<0)
```

三个优点： 1. **正区间梯度恒为 1** → 不衰减，深网络能训（这是它取代 sigmoid 的核心原因）； 2. **计算只有一次比较**（实测比 sigmoid 快约 2 倍，且 NPU/GPU 有专门硬件）； 3. **稀疏激活**（约一半神经元输出 0）→ 表征更解耦、有效计算量下降。

三个问题： 1. **死亡 ReLU**（最严重，见 C82）； 2. **非零中心**（和 sigmoid 一样的毛病）； 3. **在 0 处不可微**——实践中无所谓（框架取次梯度 0 或 1），理论分析时要注意。

C82 ★★★ 死亡 ReLU 怎么产生？怎么检测？怎么治？

A：

产生机制：如果某个神经元的输入被推到恒负区间（例如学习率过大、初始 bias 太负、或者数据分布偏移），它的梯度就**恒为 0**——**而且再也不会恢复**，因为它没有任何方式把输入拉回正区间。这就是"永久失活"。

实测演示（把一个神经元的 bias 人为设成 −10，训练 30 步）：

```
ReLU       参数总位移 = 0.000000    ← 完全不动，已经死亡
LeakyReLU  参数总位移 = 0.053267    ← 负区间还有 0.01 的斜率，能慢慢恢复
```

检测方法：统计每层激活的**零值比例**（前向时记录），健康网络通常 30%~60%；如果某层 **> 90% 且持续上升**，说明大量神经元死亡。

四个治法： 1. **换 LeakyReLU / PReLU / ELU**（负区间给斜率，最主要的手段）； 2. **He 初始化 + BN**（保证初始输入分布合理，`Var(W) = 2/n_in`）； 3. **降学习率 / 减小 weight decay**（大 lr 是头号诱因）； 4. **避免大的负 bias 初始化**（bias 用 0 或小的正数）。

C83 ★★ LeakyReLU / PReLU / ELU / SELU 各改了什么？

A：

激活	公式（负区间）	改了什么	代价
LeakyReLU	`αx`（α=0.01 常见）	给负区间一个固定小斜率 → 不死	多一个超参 α
PReLU	`αx`，**α 可学习**	让网络自己决定斜率	每通道一组参数，小数据易过拟合
ELU	`α(e^x − 1)`（→ −α 饱和）	负区间**平滑且饱和**，输出均值更接近 0 → 收敛更稳	含 exp（慢）；实测负区间梯度 0.022
SELU	特化的 ELU（λ、α 固定）	**自归一化**：配合 `lecun_normal` 初始化，能让每层输出保持 均值 0 方差 1	对初始化和网络结构极敏感，实际很少有人用

实测的"负区间平均梯度"（x < −2）：ReLU **0**、LeakyReLU 0.010、ELU 0.022、GELU **−0.008**、SiLU **−0.039**。

注意最后两个是负的——这说明 GELU/SiLU **非单调**（有小负值区），它们的"恢复能力"不是靠斜率，而是靠形状本身。

C84 ★★★ GELU 是什么？为什么 Transformer/ViT 都用它？

A：

```
GELU(x) = x · Φ(x)          Φ 是标准正态的累积分布函数
工程近似：0.5x(1 + tanh(√(2/π)(x + 0.044715x³)))     ← 实测最大误差仅 0.00041
```

直觉：`Φ(x)` 是"x 有多大概率大于 0"，所以 GELU ≈ **"按概率保留输入"**——可以看作 ReLU 的**平滑随机版本**（等价于对 dropout 做确定性期望）。

为什么 Transformer/ViT/BERT/GPT 用它： 1. **平滑**：处处可导，梯度连续，对深层优化友好； 2. **非单调**：在 x ≈ −0.17 附近有轻微负值（实测导数最小值 −0.0076），让"弱负激活"也能传递一点信号——比 ReLU 的硬截断更细腻； 3. **经验效果好**：BERT/GPT/ViT 用它，社区跟随；而且实验上比 ReLU 略好（幅度不大，1 个点以内）。

实测导数上界：GELU **1.129**、SiLU **1.100**、ReLU/tanh **1.0**、sigmoid **0.25**——前两者的导数略大于 1，深层叠加时要注意梯度放大（配合 LN 使用即可）。

C85 ★★★ SwiGLU / GeGLU 是什么？为什么大模型都用？参数量怎么补齐？

A：GLU 家族是**门控**结构：

```
GLU(a, b)  = a ⊗ σ(b)          ← 一路做内容，一路做"闸门"
SwiGLU(a,b)= a ⊗ SiLU(b)       ← LLaMA 系用它
GeGLU(a,b)= a ⊗ GELU(b)        ← PaLM 系用它
```

因为要**两个投影 + 一个输出投影**（共 3 个矩阵），比标准 FFN（2 个矩阵）多 50% 参数：

```
标准 FFN:      2 × (4096 × 11008) = 90.2 M
SwiGLU(同 F):  3 × (4096 × 11008) = 135.3 M   ← 多 50%
SwiGLU(F'=2/3F): 3 × (4096 × 7338) = 90.2 M   ← 基本持平 ✓
```

这就是为什么 LLaMA 的 FFN 维度是 `8/3·d`（而不是 GPT 的 `4·d`）——取 2/3 的 4d，正好抵消第三个矩阵带来的增量。这个数字推导面试很值分。

为什么大模型偏爱 GLU：乘法门控提供了**二阶交互**（`a·σ(b)` 能表达"当 b 满足条件时放大 a"），比逐元素非线性表达力更强；多篇论文报告在同样参数量下 perplexity 更低。

C86 ★★ 视觉和语言怎么选？端侧有什么特殊考虑？

A：

场景	首选	理由
CNN 检测/分割（追求速度）	**ReLU**	最省算力、NPU/GPU 硬件友好
CNN 追求精度/现代化	**SiLU 或 GELU**	ConvNeXt 用 GELU、YOLO 系新版用 SiLU
ViT / Transformer 视觉塔	**GELU**	与 BERT 系一致、平滑
LLM 的 FFN	**SwiGLU / GeGLU**	门控更强，LLaMA/PaLM 系标准
RNN / 生成模型隐层	**tanh**	零中心、有界（利于稳定性）
二分类输出	**sigmoid**	有概率意义
多分类输出	**softmax**（不是逐元素激活）	归一化到概率单纯形

端侧的三条特殊考虑（你面华为会用到）： 1. **ReLU 对量化最友好**（INT8 可直接做，且 NPU 有专门单元）；**GELU/SiLU 通常要用查表（LUT）或分段多项式近似**，会引入误差且可能不被算子支持； 2. **SiLU 比 GELU 便宜**（实测 6.8ms vs 23.9ms，因为 SiLU 只要一个 sigmoid）； 3. 端侧模型经常把 **GELU 换成 ReLU6 或 H-Swish**（MobileNetV3 的做法），用一点精度换可部署性。

C87 ★★ 激活函数和归一化的区别？（最常被混淆的一对）

A：

	激活函数	归一化（BN/LN）
作用	**引入非线性**	**调整分布（均值/方差）**
是否可学习	ReLU 无参数；PReLU 有	γ、β 可学习
是否改变表达力	**是**（没有它网络退化成线性）	否（只是仿射重参数化，但影响优化）
能否互相替代	**不能**	**不能**

常见误解：- "BN 也有非线性"——**错**，BN 是仿射变换，纯线性；- "用了 LN 就不需要激活"——**错**，LN 之后仍要有激活（Transformer 的 FFN 是 `LN → Linear → GELU → Linear`）；- "激活函数负责把值压到 0~1"——只有 sigmoid 是，ReLU/GELU **不归一化**。

C88 ★★ 五个常见误区与面试陷阱

A：

误区	正确说法
"softmax 是激活函数"	**它是归一化**（跨类做，输出和为 1）；逐元素激活是另一回事。混淆这一点在面试里很致命
"tanh 比 sigmoid 好，所以都该用 tanh"	tanh 只是解决了零中心，**梯度上界仍是 1（实测）**，深层仍会衰减；真解决是 ReLU 系
"ReLU 不可微所以有问题"	实践中框架取次梯度，**没有实际影响**；真要平滑用 GELU/SiLU
"非零中心只是理论问题"	实际会让收敛变慢（zigzag），是 sigmoid 被淘汰的第二大原因
"激活函数越大越好（导数 > 1）"	GELU/SiLU 的导数略大于 1（1.13/1.10），**深层会放大梯度**，必须配归一化用

最后一个陷阱最值得记：很多人看到"GELU 导数上界 1.13"会觉得"更好"——**实际上这会让残差堆叠时梯度指数放大**，所以现代架构都要配 LN 或 LayerScale（ConvNeXt 的 LayerScale 初始值 1e-6 就是防这个）。

附：复习优先级（A/B/C 三部分）

优先级	题目
必须能讲（A 部分 17 题）	A1 A2 A3 A7 A13 A14 A23 A24 A31 A32 A33 A39 **A46 A58 A62 A66 A68**
必须能推（B 部分 14 题）	B2 B3 B9 B12 B13 B14 B17 B18 B20 B29 B30 B37 B38 B40
必须能分清/讲清（C 部分模型篇 6 题）	**C55**（两个 DINO 同名不同物）**C56**（自蒸馏为什么防坍塌）**C58**（DINOv2 改进）**C62**（ViT 为什么不能直接做密集预测）**C63**（DPT reassemble+fusion）**C65**（尺度不变损失）**C69**（DA-V2 核心改动）**C71**（三阶段流程）
必须能讲（C 部分激活 4 题）	**C80**（sigmoid 为什么死：导数 0.25）**C82**（死亡 ReLU 三治）**C84**（GELU 非单调）**C85**（SwiGLU 的 2/3 补齐）
必须能算（C 部分 10 题）	**C13**（输出尺寸）**C14**（感受野）**C15**（参数量/FLOPs）**C18**（BN/LN/GN）**C20**（残差梯度）**C29**（bottleneck 参数）**C38**（SGD vs Adam）**C45/C46**（指标口径）**C51**（预处理一致性）
能讲 1 分钟（A 其余）	A4–A6 A8–A12 A15–A22 A25–A30 A34–A38 A40–A45
能讲 1 分钟（B 其余）	B1 B4–B8 B10 B11 B15 B16 B19 B21–B28 B31–B36 B39 B41–B45
能讲 1 分钟（C 其余）	C1–C12 C16 C17 C19 C21–C28 C30–C37 C39–C44 C47–C50 C52–C54 C57 C59–C61 C64 C66–C68

现场最值钱的七个"能算"（面试官会让你当场推）

1. **卷积输出尺寸** `O = floor((I + 2P − K)/S) + 1`（C13）
2. **感受野递推** `RF_l = RF_{l−1} + (K_l−1)·Π S_i`，三个 3×3 = 一个 7×7 的感受野，但参数只有 55%（C14）
3. **参数量对比** 标准 3×3(256→256) = 589,824 vs 深度可分离 = 67,840（约 1/8.7）（C15）
4. **bottleneck 省算力** 589,824 → 69,632（约 1/8.5）（C29）
5. **softmax+CE 的梯度** `∂L/∂z = p − y`（B3）
6. **为什么除 $\sqrt{d_k}$** `Var[q·k] = d_k`（B38）

7. **残差为什么救梯度** `∂L/∂x_l = ∂L/∂x_L·(1 + ∂ΣF/∂x_l)`，那个 1 是恒等路径（C20）

B/C 两部分最容易被追问的三个"为什么"

- softmax 为什么减 max（B2）→ 上溢成 inf 再除得 nan，数学等价
- kmeans 为什么只是局部最优（B30）→ 非凸 + 坐标下降，附实测 20 个种子 7 次失败
- BN 为什么 CV 默认用（C18）→ 卷积核共享让通道统计量稳定，同时平滑损失曲面